

Contents

Building Search	1
A conceptual guide for engineers, using DruSearch as a case study	1
Preface	1
Table of Contents	1
Part I — The Problem	3
Chapter 1: What Search Really Is	5
The three jobs of search	5
Relevance is not a property of a document	5
What "good" looks like	6
Chapter 2: Why Keywords Are Not Enough	7
Problem 1: Vocabulary mismatch	7
Problem 2: Word importance	7
Problem 3: User intent and behavior	8
What we are about to build	8
Part II — Retrieval: Finding Candidates	9
Chapter 3: Lexical Retrieval and BM25	11
The inverted index	11
Tokenization and analysis	12
TF-IDF: the original idea	12
BM25: the practical winner	12
Why BM25 is still good	13
Chapter 4: Embeddings — Turning Words into Numbers	15
A vector is just a list of numbers	15
What an embedding is	15
What "the model" is	15
How are these vectors actually trained?	16
What you can do with embeddings	16
Embeddings are not magic	17
Where embeddings live in the system	17
Chapter 5: Vector Search and Approximate Nearest Neighbors	19
The naive approach	19
Approximate nearest neighbors	19
How HNSW works (roughly)	20
What it costs	20
The vector search API	20

kNN: the operation, not the algorithm	20
What you actually retrieve	21
Chapter 6: Hybrid Retrieval and Rank Fusion	23
The wrong way: sum the scores	23
The right way: rank fusion	23
Why this is so effective	24
What hybrid retrieval looks like in practice	24
Other fusion approaches (briefly)	24
A note on filtering vs. retrieval	24
Part III — Ranking: Ordering What You Found	27
Chapter 7: Why Retrieval Order Is Not Good Enough	29
Retrieval optimizes for recall; ranking optimizes for precision	29
Retrieval cannot see the things that matter most	29
The shape of the solution	30
Chapter 8: Features — How a Model Sees a Result	31
The shape of a feature vector	31
Static features versus interaction features	31
Priors	32
Smoothing: why naive ratios are dangerous	32
Encoding categorical features	32
Feature parity: the highest-risk correctness boundary	32
Chapter 9: Labels — Teaching the Model What "Good" Means	35
The dream and the reality	35
Source 1: human relevance judgments	35
Source 2: behavioral labels (clicks, purchases)	36
Source 3: model-generated labels (teacher models)	36
Combining label sources	36
Sample weights	36
Chapter 10: The Click Trap — Position Bias	39
The trap	39
Where position bias comes from	39
Symptoms of position-bias collapse	40
How to escape the trap	40
A recurring lesson	40
Chapter 11: Learning to Rank	43
Three ways to formulate ranking as ML	43
Gradient-boosted decision trees	44
What training looks like	44
Splits matter more than you think	45
Avoiding overfitting	45
Chapter 12: Teacher Models and Distillation	47
The motivation	47
Distillation	47
The BGE teacher	48
When (and when not) to use a teacher	48
The serving-vs-training divide	48

Part IV — Behavior and Personalization	49
Chapter 13: Events — The Feedback Loop	51
What an event is	51
The event log	51
Event quality	52
Sessions	52
Async event ingestion	52
What events become	52
Chapter 14: Personalization	55
Levels of personalization	55
The cold-start problem	55
Online vs. offline user features	56
The privacy and fairness dimension	56
The "filter bubble" pitfall	56
Part V — Evaluation and Operations	59
Chapter 15: Did It Get Better? Offline Evaluation	61
Two kinds of evaluation	61
Ranking metrics	61
Building an offline eval set	62
RRF as a baseline	63
Promotion gates	63
Chapter 16: The Two Worlds — Online and Offline	65
The two worlds	65
Examples of the divide	65
Feature stores	65
The training/serving skew problem	66
What this means for engineering practice	66
Chapter 17: Latency, Failure, and Graceful Degradation	67
The latency budget	67
Parallelism and pipelining	67
Timeouts	68
Circuit breakers	68
Graceful degradation	68
Caching	69
Load and capacity planning	69
Chapter 18: Promotion — From Trained to Served	71
Models are versioned data	71
MLflow	71
The promotion gate	72
Promotion mechanics	72
Rollback	72
A/B testing	73
Chapter 19: Observability	75
The three pillars	75
Prometheus and Grafana	75
What to alert on	76
Logs as a debugging tool	76

Service-level objectives	76
What good looks like	76
Part VI — Putting It Together	79
Chapter 20: The Full System	81
The cold path: building the index and the model	81
The hot path: handling a query	82
The feedback loop	82
What this book has and has not covered	83
A final thought	84
Glossary	84

Building Search

A conceptual guide for engineers, using DruSearch as a case study

Preface

If you have built web applications, written a database query, or wired a frontend to a backend, you already know more than enough to understand how a modern search system works. What you probably do not have — and what this book tries to give you — is a clear mental model of the *machine learning* parts: vectors, embeddings, rerankers, training labels, evaluation, and the strange feedback loops that come with all of them.

This is not a book about a particular library or framework. It is a book about *concepts*. Each chapter introduces an idea, explains the problem it solves, defines every term it uses, and shows where the idea fits in a real system. The “real system” we will refer to throughout is **DruSearch**, a small e-commerce search stack, but the concepts apply to any search product: a documentation search, an internal company knowledge base, a product catalog, a code search, a music recommender, or a question-answering chatbot.

The book is meant to be read in order the first time, then used as a reference afterward. Each chapter assumes you have read the ones before it.

A note on language: machine learning has a bad habit of using ten-syllable words for simple ideas. When we encounter such a word, we will call it out, define it plainly, and then keep using it — because you will see it everywhere else. By the end, none of those words should feel mysterious.

Table of Contents

Part I — The Problem

1. What Search Really Is
2. Why Keywords Are Not Enough

Part II — Retrieval: Finding Candidates 3. Lexical Retrieval and BM25 4. Embeddings: Turning Words into Numbers 5. Vector Search and Approximate Nearest Neighbors 6. Hybrid Retrieval and Rank Fusion

Part III — Ranking: Ordering What You Found 7. Why Retrieval Order Is Not Good Enough 8. Features: How a Model Sees a Result 9. Labels: Teaching the Model What “Good” Means 10. The Click Trap: Position Bias 11. Learning to Rank 12. Teacher Models and Distillation

Part IV — Behavior and Personalization 13. Events: The Feedback Loop 14. Personalization

Part V — Evaluation and Operations 15. Did It Get Better? Offline Evaluation 16. The Two Worlds: Online and Offline 17. Latency, Failure, and Graceful Degradation 18. Promotion: From Trained to Served 19. Observability

Part VI — Putting It Together 20. The Full System

Part I — The Problem

Chapter 1: What Search Really Is

When a user types "running shoes" into a search box, what are they actually asking for? It feels like an obvious question. They want to see running shoes. But pause on it for a moment, because the obvious answer hides three different problems that the rest of this book is about.

The three jobs of search

Every search system, whether it is searching a product catalog, a corpus of medical papers, a code repository, or the open web, does three jobs:

1. **Understand the query.** What does the user mean by these words? "Running shoes" probably means athletic footwear designed for running. But "apple" might mean the fruit, the company, a record label, or a person's last name. Even the simple query "shoes for my husband" carries hidden information — the shopper is buying for someone else, probably wants men's shoes, and is unlikely to want anything labeled "women's."
2. **Find candidate results.** Out of millions of possible documents, products, or files, which ones are even worth considering? This step is called **retrieval**. The goal of retrieval is not to find the perfect answer; it is to find a small set (typically a few hundred) that almost certainly contains the perfect answer.
3. **Rank the candidates.** Given those few hundred candidates, which ten should we put at the top of the page? This step is called **ranking**. A good ranker puts the most relevant, useful, or likely-to-be-clicked results first.

These three jobs sound like they could be one job, but they are not, and conflating them is the most common mistake people make when first thinking about search.

The reason they are separate is **scale**. If you have 10,000 products, you can afford to score each one carefully against every query. If you have 10 million, or 10 billion, you cannot. Retrieval has to be cheap and fast, because it runs over everything. Ranking can afford to be expensive, because it only runs over a few hundred candidates.

Relevance is not a property of a document

Here is the second non-obvious point: **relevance is not a property of a document; it is a property of a (query, document, user, context) tuple.**

The same product can be highly relevant to one query and useless for another. The same query, asked by two different people, can have two different best answers. "Coffee" at 7 a.m. on a phone in San Francisco probably means "coffee shop near me." "Coffee" at 11 p.m. on a desktop might mean recipes, or beans for sale, or an article about caffeine.

This sounds like a small philosophical point, but it has huge engineering consequences. It means we cannot just label each product with a "quality score" and sort by that. The same product needs different scores

depending on who is searching, what they typed, and what they have done before. That is what features and ranking models are for, and we will spend most of Part III on them.

What “good” looks like

Before we can build a search system, we need a rough idea of what success means. A useful working definition: **a search result is good if the user finds what they wanted with the least effort.** A page of search results is good if, on average, its users do.

That definition is operational. It points us toward what we should measure: did users click? Did they buy? Did they refine their query because the first result was wrong? Did they leave the site? We will get to the formal versions of these measurements (NDCG, recall, click-through rate) in later chapters.

For now, hold three facts in your head:

- Search has three sub-problems: query understanding, retrieval, and ranking.
- Relevance depends on the user and context, not just the document.
- “Good” search means the user finds what they wanted quickly.

The rest of the book is about how to actually build that.

Chapter 2: Why Keywords Are Not Enough

The simplest possible search system is keyword matching: store a list of documents, and when a query comes in, return every document that contains the query's words. This is how `grep` works. It is also how most early search engines worked, and it is still a useful starting point.

Keyword matching is fast, cheap, and often surprisingly good. But it has three problems that, together, are why modern search uses everything else in this book.

Problem 1: Vocabulary mismatch

People do not type the words that are in your documents. They type the words that are in their heads.

A user looking for a "couch" might land on a product page that uses the word "sofa." A user searching for "rain jacket" might miss a "windbreaker" that would have been perfect. A user searching for "low-cost laptop" will not find a product whose description says "budget-friendly notebook computer."

This is called the **vocabulary mismatch problem**, and it is everywhere. Synonyms, paraphrases, abbreviations, brand names versus generic names, technical jargon versus everyday language — all of these create a gap between what the user types and what your data says.

You can patch some of this with synonym dictionaries, but synonym lists are brittle. They miss the long tail of phrasings, they go stale, and they cannot capture *similarity in meaning* — the idea that "running shoes" and "trainers for jogging" are about the same thing even though no two words overlap.

What we want is a way to compare meanings, not just spellings. That is what **embeddings** (Chapter 4) give us.

Problem 2: Word importance

Not all words in a query carry equal weight. In the query "the best red running shoes for women," the words "best," "red," "running," "shoes," and "women" matter a lot. The word "the" matters almost not at all. The word "for" matters somewhere in between.

Worse, some words matter a lot in one query and not at all in another. "Apple" is critical in "apple iphone" and almost noise in "apple pie recipe."

A naive keyword matcher treats all matching words equally. We need a way to say "this query word is rare and meaningful, that one is common filler." That is what **TF-IDF** and **BM25** (Chapter 3) do.

Problem 3: User intent and behavior

Two products can both contain the words "running shoes" in their descriptions. One is a popular, well-reviewed athletic shoe sold by a known brand. The other is a generic listing posted by a third-party seller, no reviews, no sales. Both match the query equally on the keywords. Which should rank higher?

Keyword matching has nothing to say about this. It does not know which products are popular. It does not know which ones get clicked. It does not know whether *this particular user* tends to buy from premium brands or always picks the cheapest option.

To answer "which product should rank higher?" we need to combine many signals: keyword match, semantic match, popularity, user history, prices, reviews, freshness, and dozens more. That combination is the job of a **ranking model** (Chapter 11).

What we are about to build

The rest of the book is, essentially, a layered answer to these three problems:

- For vocabulary mismatch, we add **dense vector retrieval** (Part II).
- For word importance, we use **BM25** (Chapter 3).
- For combining signals to pick the best result, we use **learning to rank** (Part III).
- For knowing which results users actually like, we collect **events** and use them as feedback (Part IV).
- For checking whether any of this is actually working, we use **evaluation** (Chapter 15).

DruSearch, the system we will use as a running example, does all of these things. By the end of the book, the architecture diagram in its README should read like a sentence in a language you speak.

Part II — Retrieval: Finding Candidates

Chapter 3: Lexical Retrieval and BM25

Lexical means "having to do with words." **Lexical retrieval** is just a fancy term for what `grep` does: matching the actual words of the query against the actual words in your documents. Despite its simplicity, lexical retrieval is still one of the strongest tools in the search toolbox, and it is the foundation that everything else is layered on top of.

This chapter explains how lexical retrieval works under the hood, why a scoring function called **BM25** has dominated it for thirty years, and what BM25 is actually computing.

The inverted index

Suppose you have a million product descriptions and a query: "red running shoes." How do you find every product whose description contains all three words?

You could scan every product. With a million products, that is too slow.

Instead, you build an **inverted index**. The "forward" view is: each document has a list of words. The inverted view flips that around: each word has a list of documents.

forward:

```
doc_1: ["red", "running", "shoes", "nike", "athletic"]
doc_2: ["blue", "running", "tights", "compression"]
doc_3: ["red", "dress", "shoes", "leather"]
```

inverted:

```
"red":      [doc_1, doc_3]
"running":  [doc_1, doc_2]
"shoes":    [doc_1, doc_3]
"tights":   [doc_2]
"leather":  [doc_3]
...
```

To find documents that contain all of "red," "running," and "shoes," you intersect the three lists: `[doc_1, doc_3] ∩ [doc_1, doc_2] ∩ [doc_1, doc_3] = [doc_1]`.

This is the structure that powers **Lucene**, the open-source library that underlies **Elasticsearch**, **OpenSearch**, **Solr**, and most other production search engines. (DruSearch uses OpenSearch, which is a fork of Elasticsearch maintained by Amazon and the open-source community. They are interchangeable for our purposes.)

The inverted index is what makes lexical search fast. Even with billions of documents, finding candidates that match a few words takes milliseconds.

Tokenization and analysis

Before you can build an inverted index, you need to decide what counts as a “word.” This step is called **tokenization**, and the broader process of preparing text for indexing is called **analysis**.

A typical analysis pipeline does several things:

1. **Lowercasing.** “Red” and “red” should match.
2. **Punctuation handling.** “shoes.” should match “shoes.”
3. **Stopword removal.** Common words like “the,” “a,” “of,” “to” are sometimes dropped because they appear everywhere and rarely affect meaning. (Modern systems often keep them because they sometimes do matter — “the who” is a band, “to be” is a quote.)
4. **Stemming or lemmatization.** “Running,” “ran,” and “runs” might all be reduced to “run” so they all match. Stemming is a crude rule-based version of this; lemmatization is a smarter linguistic version.
5. **Synonym expansion.** “Sofa” might be expanded to also index as “couch.”

Different fields can use different analyzers. In DruSearch, the product `title`, `description`, and `bullets` (the bullet points in a product listing) all use a custom English-language analyzer. The `brand` and `color` fields are indexed as **keywords** — meaning they are not analyzed at all, just stored as exact strings, because you usually want exact matches on brand names.

TF-IDF: the original idea

If you find ten documents that all contain “red running shoes,” how do you decide which to show first? You score them. The classic scoring function is **TF-IDF**, which stands for **term frequency, inverse document frequency**.

The intuition is two-part:

- **Term frequency (TF):** a document that mentions “running” five times is probably more about running than one that mentions it once. So we score documents partly based on how often the query terms appear.
- **Inverse document frequency (IDF):** a query word that appears in almost every document is not very informative. “The” appears everywhere; “athleisure” appears in a few hundred. The rare word should weigh more heavily in the score. IDF is, roughly, “one over how many documents contain this word.”

Multiply them and sum across query terms, and you get a TF-IDF score. Higher is better.

TF-IDF works, but it has a subtle problem: term frequency grows linearly. A document mentioning “running” twenty times scores twenty times higher than one mentioning it once. In practice, the second mention of a word matters a lot less than the first, and the twentieth probably does not matter at all.

BM25: the practical winner

BM25 stands for **Best Matching, attempt 25** — it really is the twenty-fifth iteration of an algorithm developed at the City University of London in the 1970s and 80s. It is the standard ranking function used by Lucene and almost every production lexical search engine today.

BM25 takes the same TF-IDF intuition and fixes two things:

1. **Term frequency saturates.** The first occurrence of “running” in a document matters a lot. The second matters somewhat. The tenth barely matters at all. BM25 uses a curve that flattens out, so additional occurrences contribute less and less.
2. **Document length is normalized.** A long document is more likely to contain any given word just by chance. BM25 penalizes long documents that match, and rewards short documents that match — relative to the average document length in the corpus.

The result is a scoring formula with two tunable parameters, conventionally called **k1** (controls TF saturation) and **b** (controls length normalization). You will rarely change them; the defaults work well almost everywhere.

For our purposes, the important things to remember about BM25 are:

- Higher is better, but the absolute number is not directly comparable across queries.
- It is fast — it takes microseconds per document because the inverted index does the hard work.
- It rewards documents that contain rare query words and penalizes very long documents.

In DruSearch, BM25 runs across multiple fields with different weights. The product title is weighted twice as heavily as the description, because a match in the title is much stronger evidence of relevance than a match buried in a description. Category names are weighted somewhere in between. This is called **multi-field BM25** or, in OpenSearch syntax, a `multi_match` query.

Why BM25 is still good

It is tempting to dismiss BM25 as the "old" way of searching now that we have neural networks. Resist that temptation. BM25 is:

- Extremely fast.
- Predictable and explainable. If a document scores high, you can point to the words that made it score high.
- Good at exact matches. If someone searches for a specific product code, model number, or proper noun, BM25 will find it perfectly. Vector search often will not.
- Cheap to build and maintain. The inverted index is a well-understood data structure.

Modern search systems do not replace BM25; they add to it. The next chapters introduce embeddings and vector search, and Chapter 6 explains how to combine them with BM25 to get the best of both.

Chapter 4: Embeddings — Turning Words into Numbers

The vocabulary mismatch problem from Chapter 2 — that "couch" and "sofa" mean almost the same thing but share zero letters — cannot be solved by any amount of clever lexical matching. To fix it, we need a way to represent **meaning** rather than **spelling**.

That representation is called an **embedding**, and understanding it is the single most important step in moving from "I know how to use a search engine" to "I know how a search engine works."

A vector is just a list of numbers

A **vector** is a list of numbers. That is all. $[3, 1, 4, 1, 5, 9, 2, 6]$ is an 8-dimensional vector. $[0.42, -0.18, 0.07, \dots, 0.31]$ with 384 numbers in it is a 384-dimensional vector. The "dimension" is just how many numbers are in the list.

When we say a vector lives in "high-dimensional space," all that means is that there are a lot of numbers in the list. We talk about points in 2D or 3D space because we can visualize them. A 384-dimensional point is no different mathematically — it just cannot be drawn.

What an embedding is

An **embedding** is a vector that represents the meaning of something — a word, a sentence, a product, an image, a song — in such a way that *similar things have similar vectors*.

The word "run" might be represented by a 384-number vector. The word "jog" gets its own vector. The phrase "athletic footwear" gets another. The trick is that these vectors are computed by a **model** (more on what that means in a moment) trained so that the vectors for things with similar meanings end up close to each other in this high-dimensional space.

"Close" here has a precise meaning. The two most common ways to measure closeness between vectors are:

- **Cosine similarity**: how much the two vectors point in the same direction. Two vectors pointing the same way have cosine similarity 1.0; perpendicular vectors have similarity 0; opposite vectors have similarity -1.0.
- **Euclidean distance** (sometimes called L2 distance): the straight-line distance between the two vector "points," computed by the same formula you learned in geometry class, generalized to many dimensions.

For text, cosine similarity is the most common choice. DruSearch uses it.

What "the model" is

When we say "a model produces embeddings," what does that mean concretely?

A **model**, in the machine-learning sense, is a function that takes input and produces output, where the function's behavior was learned from data rather than written by hand. The function itself is parameterized by millions or billions of numbers (called **weights** or **parameters**), and those weights were determined during a process called **training**.

A text-embedding model takes a piece of text as input and produces a fixed-length vector as output. The same input always produces the same output. Two pieces of text with similar meanings produce vectors that are close to each other.

You do not need to understand how the model works internally to use one. You can treat it as a function: `embed("running shoes")` returns a 384-dimensional vector. Done. The internals are a **transformer neural network**, but for the purposes of this book, the model is a black box that converts text to vectors.

DruSearch uses a model called **BGE-small-en-v1.5**. The name is a string of identifiers:

- **BGE**: BAAI General Embeddings. (BAAI is the Beijing Academy of Artificial Intelligence, the lab that released the model.)
- **small**: the smallest size of the BGE family — fewer parameters, faster, slightly less accurate.
- **en**: English-language.
- **v1.5**: version.

It produces 384-dimensional vectors. The exact dimension is a design choice the model's authors made. Smaller dimensions mean faster computation and less storage but slightly less expressive embeddings. 384, 768, and 1536 are common sizes.

How are these vectors actually trained?

You do not need to know how to train an embedding model in order to use one — and most teams use pre-trained off-the-shelf models. But a one-paragraph sketch of the training process helps demystify the result.

Embedding models are trained on enormous collections of text — the open web, books, Wikipedia, code, scientific papers — using an objective like: "given a question and its correct answer, make their vectors close. Given a question and a random unrelated paragraph, make their vectors far apart." Repeat across billions of examples, and the model gradually learns to map text into a space where semantic similarity corresponds to geometric closeness. This idea is called **contrastive learning**, and the resulting vectors emerge with a remarkable property: they capture meaning even for inputs the model has never seen before.

The model does not have a hand-coded notion of what "running shoes" or "couch" mean. It learned it from patterns in data. This is both why embedding models are powerful and why they are hard to reason about: the meaning of dimension #137 in the output vector is, in general, not interpretable. The vector as a whole encodes meaning; no individual number does.

What you can do with embeddings

Two superpowers come from this representation:

1. **Semantic similarity**: compare two pieces of text by computing the cosine similarity between their vectors. "Sofa" and "couch" will be close. "Sofa" and "philosophy" will be far. Crucially, this works without any synonym dictionary; the model learned the relationship from context in its training data.
2. **Semantic search**: index the embedding vectors of every document in your catalog. At query time, embed the query, and find the documents whose vectors are closest to it. That is **vector search**, and it is the topic of the next chapter.

Embeddings have other uses — clustering similar items, recommending related products, detecting near-duplicates — but for search, semantic search is the headline feature.

Embeddings are not magic

A crucial caveat. Embedding models are trained on general text, and the geometry of their vector space reflects the patterns of that training data. They are very good at capturing broad semantic similarity. They are usually worse than BM25 at:

- **Exact matches.** A specific model number like "iPhone 15 Pro Max 256GB" is best handled by lexical search. The embedding might place it near other phones, but BM25 will pinpoint it.
- **Rare or domain-specific terms.** Medical jargon, legal terminology, or internal company codes that did not appear much in the training data can have embeddings that are not very informative.
- **Negation and small but important words.** "Running shoes for men" and "running shoes not for men" will produce embeddings that are surprisingly close, because most of the words are the same and the model does not always learn to flip meaning on a single word.

This is why production search systems do not use embeddings *instead of* BM25. They use them *in addition to* BM25. Chapter 6 is about how to combine them.

Where embeddings live in the system

In DruSearch, embeddings are produced by a separate small service called the **embedder sidecar**. The Go API service does not load the embedding model itself; instead, it sends a small HTTP request to the sidecar with the text and gets back the vector.

This separation is deliberate. The embedding model is large (hundreds of megabytes to gigabytes), it has heavy dependencies (deep-learning libraries like **PyTorch**), and it is best run with GPU acceleration when available. The Go API stays small, fast, and free of those dependencies. The same sidecar is used both at *index time* (when product titles are converted to vectors and stored) and at *query time* (when each user query is converted to a vector). This guarantees that products and queries are embedded by the same model, so their vectors are comparable.

A common bug in homegrown semantic search systems: indexing with one version of a model and querying with a different one. The vectors are then in different spaces and similarity scores become noise. Treat the embedding model as a versioned dependency.

Chapter 5: Vector Search and Approximate Nearest Neighbors

We now have query embeddings and product embeddings. The next problem is mechanical: given a query vector, how do you efficiently find the closest product vectors out of millions?

The naive approach

The straightforward way: compute the cosine similarity between the query vector and *every* product vector, then sort by that score. This is called a **brute-force** or **exhaustive** search.

The math is simple. If you have one million products and 384-dimensional vectors, each query requires one million dot products of 384 numbers each. That is roughly 400 million arithmetic operations per query. On a modern CPU, this takes hundreds of milliseconds. For a search system that needs to respond in 50 milliseconds, that is too slow.

For small catalogs (say, under 100,000 items), brute-force vector search works fine. Beyond that, you need something cleverer.

Approximate nearest neighbors

The clever something is called **ANN**, which stands for **Approximate Nearest Neighbors**. The word “approximate” is the key. We give up on finding the *absolute* closest vectors, and settle for finding *very close* ones with high probability. In exchange, we get speedups of 10x, 100x, or more.

This is a worthwhile trade because:

- The user does not care whether result #1 is *the* closest by 0.00001. They care that the top results are good.
- Embeddings are noisy approximations of meaning anyway. Looking for the mathematically exact closest vector is a kind of false precision.
- The speedup is enormous. ANN can return top results in under a millisecond on a million-item index.

Several ANN algorithms exist; you will encounter their names in product documentation:

- **HNSW** (Hierarchical Navigable Small World) — currently the most popular, used by DruSearch via OpenSearch.
- **IVF** (Inverted File index) — partitions vectors into clusters and only searches the relevant clusters.
- **PQ** (Product Quantization) — compresses vectors to save memory.
- **ScaNN**, **FAISS**, **Annoy** — names of libraries that implement these algorithms.

You do not need to understand the algorithms in detail. You need to understand what ANN gives you and what it costs.

How HNSW works (roughly)

A one-paragraph mental model of HNSW, since you will see it referenced everywhere:

Imagine a graph where every product vector is a node, and edges connect vectors that are close to each other. To find vectors close to a query, you start at some node and walk along edges, always moving toward closer vectors, until you cannot get any closer. This is a **greedy graph traversal** — at each step, you take whichever neighbor brings you nearest to the query.

That alone would be slow because you might start far from the answer. HNSW fixes this with a **hierarchy**: a top layer with very few, far-apart vectors, then progressively denser layers below. You start at the top, find the closest vector there, drop down a layer, navigate from there, drop down, and so on. By the time you reach the bottom (densest) layer, you are already close to the answer and only need to walk a few steps.

The result is sub-millisecond search over millions of vectors.

What it costs

ANN gives you speed in exchange for three costs:

1. **Build time and memory.** The HNSW graph has to be constructed, and it lives in memory. For a million 384-dim vectors, the index might be a few gigabytes of RAM.
2. **Approximation error.** Sometimes the algorithm misses the true nearest neighbor. In practice this is rare (single-digit percent at default settings), and the missed result is usually only slightly worse than the true best.
3. **Tuning parameters.** Every ANN algorithm has knobs. HNSW has `M` (graph connectivity) and `ef_construction` and `ef_search` (search exhaustiveness). Defaults are usually fine; advanced tuning trades recall for speed.

The vector search API

In a production system, you usually do not implement HNSW yourself. You hand vectors and queries to a **vector database** or a search engine that supports vectors. Some options:

- **OpenSearch / Elasticsearch** with the `knn_vector` field type (DruSearch's choice).
- **pgvector**, an extension for PostgreSQL.
- **Pinecone, Weaviate, Qdrant, Milvus, Vespa** — purpose-built vector databases.
- **FAISS** — a library you embed in your own service.

The choice depends on scale, existing infrastructure, and whether you need vector search alongside lexical search (in which case using your existing search engine is convenient).

In DruSearch, vectors are stored in the same OpenSearch index as the lexical fields. A single index has both the BM25 fields and the `title_vec` field. This makes hybrid retrieval (the next chapter) easier to implement.

kNN: the operation, not the algorithm

You will hear the term **kNN** thrown around. It stands for **k-Nearest Neighbors**: given a query and a value `k`, return the `k` items closest to the query. That is the *operation*, distinct from the *algorithm* used to perform it. ANN is one fast (approximate) way to do kNN. Brute-force is the exact way. When DruSearch's documentation says the API runs a kNN search, it means "the operation of finding the nearest neighbors of the query vector," which OpenSearch performs using HNSW under the hood.

What you actually retrieve

A vector search returns a ranked list of items, each with a similarity score. In DruSearch, a kNN search over the product catalog returns the top 100 products whose `title_vec` is closest to the query embedding, along with their cosine similarity scores.

These are the *semantic* candidates. The next step is combining them with the *lexical* candidates from BM25, which is what Chapter 6 is about.

Chapter 6: Hybrid Retrieval and Rank Fusion

Lexical retrieval (BM25) is good at exact matches and rare terms. Semantic retrieval (vector search) is good at meaning and synonyms. Each has weaknesses the other covers. The natural conclusion is to use both. Doing so is called **hybrid retrieval**.

The question is how. If BM25 returns one ranked list and vector search returns another, how do you combine them into a single list?

The wrong way: sum the scores

The first thing most people try is: just add the two scores together. Take the BM25 score and the cosine similarity, and sum.

This does not work, for a simple reason: the two scores are on completely different scales.

- BM25 scores typically range from 0 to maybe 50, with the magnitude depending on how many query terms matched, the length of the documents, the IDF of the words, and so on. They are not normalized.
- Cosine similarity ranges from -1 to 1, but for sentence embeddings it is usually crammed into 0.3 to 0.9 in practice.

Adding 25 to 0.7 gives 25.7. The vector score is now noise. You could try to scale them, but the relative weighting depends on the query, the corpus, and the model — there is no universal multiplier that works.

The right way: rank fusion

A better idea: ignore the absolute scores. Use only the *ranks* — the order of the results. If a document is in the top 5 of both lists, that is strong evidence it is relevant, regardless of what the underlying scores were.

The standard algorithm for this is **Reciprocal Rank Fusion**, abbreviated **RRF**. It is a tiny formula with an outsized impact in production search systems.

The formula:

$$\text{RRF_score}(d) = \text{sum over all ranked lists of } 1 / (k + \text{rank}(d))$$

Where:

- d is a document that appears in at least one of the ranked lists.
- $\text{rank}(d)$ is the position of d in a particular list (1 for the top result, 2 for the second, and so on). If d does not appear in a list, that list contributes nothing.
- k is a small constant, typically 60. Its purpose is to dampen the contribution of the top ranks so that the difference between rank 1 and rank 2 is not catastrophically larger than between rank 50 and rank 51. The value 60 is from the original 2009 paper that introduced RRF; it works well and almost everyone uses it.

So a document at rank 1 in BM25 and rank 3 in vector search gets $1/(60+1) + 1/(60+3) = 0.0164 + 0.0159 = 0.0323$. A document only in BM25 at rank 1 gets 0.0164 . The fusion rewards documents that appear high in both lists.

Why this is so effective

RRF has three nice properties:

1. **Scale-free.** It does not matter what the underlying score scales are; only ranks matter. You can fuse any number of ranked lists from any sources.
2. **Robust.** If one retrieval system has a bug or a strange result, its mistake is bounded. The bad document might be at the top of one list but if it is not in the other list, it gets a modest combined score.
3. **Trivially cheap.** The formula is a few additions per document. It runs in microseconds.

What hybrid retrieval looks like in practice

The DruSearch hybrid retrieval flow is representative:

1. Embed the query (call to the embedder sidecar).
2. In parallel:
 - Run a BM25 query over the inverted index. Get the top 100 results with their ranks.
 - Run a kNN query over the vector index. Get the top 100 results with their ranks.
3. Fuse the two lists with RRF using $k=60$. Now you have a single combined list, sorted by RRF score.
4. Take the top N (say, 100) for further processing.

Those top 100 are now the **candidate set** that the ranker will work on. They are highly likely to contain the truly best results, and they are a small enough set to score with an expensive ranking model.

Other fusion approaches (briefly)

RRF is the most common approach, but you should know that others exist:

- **Linear weighted fusion** with score normalization. Normalize each score list (e.g., min-max scaling), then $\text{combined_score} = \alpha * \text{normalized_bm25} + (1-\alpha) * \text{normalized_vector}$. Tunable, sometimes better than RRF, but you have to pick α and the right scaling.
- **Learned fusion.** Train a small model that takes both scores plus other signals and predicts relevance. Powerful, but adds complexity. In practice, this is what the LTR reranker (Chapter 11) effectively does — the BM25 and vector scores both become *features* that a ranking model uses, alongside many others.

For getting the *candidate set*, RRF is hard to beat for the simplicity-to-quality ratio. For *final ordering*, ranking models do better. That is the natural division of labor we will use.

A note on filtering vs. retrieval

In product search, you often have **hard constraints**: only show items in stock, only show items under \$200, only show items in a particular category. These should usually be applied as **filters** on top of retrieval, not built into the scoring. Filters are simple, predictable, and can use database indexes for speed.

But beware applying too many hard filters too early. If a user types "shoes for my husband," it is tempting to filter out every product not labeled "men's." But your catalog might have unisex shoes, mislabeled items, or items with no gender field at all. A hard filter can produce zero results or hide good ones. The safer approach is a **soft signal**: pass gender as a *feature* into the ranker, and let the ranker learn to prefer matching genders without strictly excluding others. DruSearch uses this approach for gender — it is a soft signal derived from

text and category during ingestion, used to *boost* matches and *demote* mismatches without filtering anyone out.

The general lesson: hard filters are good for hard constraints (like "in stock"). Soft signals are better for fuzzy preferences (like "probably men's shoes"). Use the right tool for each.

Part III — Ranking: Ordering What You Found

Chapter 7: Why Retrieval Order Is Not Good Enough

By the end of Part II, we have a fast, robust way to produce a candidate set: a few hundred documents that are very likely to contain the best results. They are even *roughly* ordered by something reasonable (RRF score). Why not just show them in that order and call it done?

Some search systems do exactly that, and it works tolerably well. But almost any production system that cares about quality will *re-order* the candidates with a more sophisticated process called **ranking** or **reranking**. This chapter explains why.

Retrieval optimizes for recall; ranking optimizes for precision

A first useful framing. Two of the foundational metrics in search:

- **Recall:** of all the relevant items in the catalog, what fraction did we find? "Did we get everything good into our candidate set?"
- **Precision:** of the items we are showing, what fraction are relevant? "Is what we are showing the user actually good?"

Retrieval is optimized for recall. We want to be very confident that the truly best results are *somewhere* in our candidate set. Whether they are at position 3 or position 73 is less important.

Ranking is optimized for precision, especially at the top. We want the top 1, top 3, and top 10 positions to be filled with the best results. Whether position 73 is a slightly better or slightly worse item is much less important — the user will probably never see it.

These are different jobs, and they call for different techniques.

Retrieval cannot see the things that matter most

BM25 and vector search both rank results based on the *match* between the query and the document. They do a good job of capturing whether the document is *about* the query. But many of the things that determine whether a result is *good* have nothing to do with the match itself:

- **Popularity.** If 10,000 users have bought this product after searching this query, that is strong evidence it is a good answer — and BM25 has no way to know that.
- **Quality.** A product with great reviews, lots of ratings, fast shipping, and a known brand is probably a better choice than an obscure one with no reviews — even if the keyword match is the same.
- **Recency.** For news, fashion, social media, or anything else where freshness matters, retrieval cannot tell new from old without help.
- **Personalization.** This particular user prefers minimalist designs, or has bought this brand five times before. Retrieval has no idea who is searching.

- **Price.** A user searching "cheap laptops" wants the lower-priced ones at the top; a user searching "premium laptops" wants the opposite. Retrieval cannot tell the difference.
- **Diversity.** If the top 10 results are all variations of the same product, the user is poorly served. Retrieval makes no diversity guarantee.

Each of these is a **signal**: a piece of information that should influence ranking but is not part of the basic query–document match. To use them, we need a ranking system that can combine many signals into a single ordering.

The shape of the solution

Conceptually, what we want is a function:

`score(query, document, user, context) -> number`

...that produces a single score we can sort by. The function should incorporate:

- The retrieval signals (BM25 score, vector similarity, RRF rank).
- Document signals (popularity, price, brand, category, freshness, quality).
- User signals (history, preferences).
- Query–document interaction signals (does the brand match a brand the user likes? does the gender match the implied gender of the query?).
- Context signals (time of day, device, location — though DruSearch does not use these).

Writing this function by hand is hopeless. There are too many signals and too many interactions between them. Hand-tuning the weight on "brand match" against the weight on "BM25 score" against the weight on "user affinity" is an enormous, unmaintainable spreadsheet.

This is the perfect setting for **machine learning**. We collect data — examples of (query, document, label) — and *learn* the function from data.

That family of techniques is called **Learning to Rank**, abbreviated **LTR**. The next several chapters introduce the pieces:

- **Features** (Chapter 8): how we describe a (query, document, user, context) tuple as a list of numbers the model can use.
 - **Labels** (Chapter 9): how we tell the model which results are good and which are not.
 - **Position bias** (Chapter 10): a key trap in defining labels from user behavior.
 - **Learning to rank** (Chapter 11): the actual training process.
 - **Teacher models** (Chapter 12): a clever way to generate good labels at scale.
-

Chapter 8: Features — How a Model Sees a Result

A machine-learning model cannot see "a product." It cannot see "a query." It can only see numbers. So the first job, before training any ranking model, is to convert each (query, document, user) candidate into a fixed-length list of numbers.

That list is called a **feature vector**. Each number in it is a **feature**. The work of designing what those numbers should be is called **feature engineering**.

This chapter explains what features look like in a search ranker, and the surprising amount of correctness work that goes into them.

The shape of a feature vector

For each candidate the ranker will score, you build a feature vector. In DruSearch, that vector currently has 29 features. A simplified view of what those features capture:

Group	Examples
Retrieval signals	BM25 score, BM25 rank, vector similarity, vector rank, RRF score
Static product properties	Popularity prior, click-through-rate prior, log of price, title length, derived gender
Query-product interaction	Whether the query brand matches the product brand, whether the query color matches, exact
User personalization	The user's affinity for the product's brand

The features are concatenated into a single vector. Every product gets a vector of the same length, in the same order. The model takes this vector as input and produces a single number — the predicted ranking score — as output.

Static features versus interaction features

Notice the structural distinction:

- **Static features** depend only on the document, regardless of query or user. "How popular is this product?" "What is its price?" These can be precomputed once per document and stored.
- **Interaction features** depend on the query, the document, and possibly the user together. "Does the query mention this product's brand?" "How well does the title cover the query terms?" These have to be computed on the fly, per (query, document, user) triple, at request time.

Static features are cheap. Interaction features are expensive. A surprising amount of search-engineering work goes into making interaction features fast enough that you can compute them on hundreds of candidates within a few-millisecond budget.

Priors

Several features in DruSearch are described as **priors**. A **prior**, in machine-learning vocabulary, is a starting estimate of something based on prior data, used in the absence of more direct evidence. Two examples:

- **Popularity prior**: a smoothed estimate of how often this product is bought or clicked across all users and queries. A product with 1,000 purchases gets a higher prior than one with 5 purchases.
- **CTR prior** (where **CTR** stands for **Click-Through Rate**): the fraction of impressions of this product that resulted in a click, smoothed so that products with very few impressions do not produce wild estimates. A product shown 1,000 times and clicked 100 times has a CTR of 10%.

Why "prior"? Because these are general, query-agnostic estimates. They tell you what to believe about a product *before* knowing the query. The ranker combines them with query-specific signals to produce a final score.

Smoothing: why naive ratios are dangerous

A subtle but important detail. Suppose a brand-new product has been shown 3 times and clicked 2 times. Its raw CTR is $2/3 = 67\%$. That is much higher than a popular product that has been shown 10,000 times and clicked 1,000 times (CTR 10%). Should the new product win?

Almost certainly not. With only 3 observations, the 67% is mostly noise; we have very little evidence. The popular product's 10% is much more reliable. To handle this, CTR priors are **smoothed** — a small fraction of a "prior expected" CTR is mixed in. With heavy smoothing, the new product's effective CTR pulls back toward the global average; with lots of data, the smoothing fades and the true rate dominates.

This pattern shows up everywhere in search and recommendation features. Whenever you compute a ratio over a small denominator, smooth it. The technical name for the simplest version is **Laplace smoothing** or **additive smoothing**. The deeper version is **Bayesian smoothing**, in which the prior is treated formally as a probability distribution. For our purposes: do not trust raw averages of small samples, ever.

Encoding categorical features

Many useful properties are categorical: brand name, color, category, gender. A model cannot consume the string "Nike" directly. There are a few common encodings:

- **One-hot encoding**: turn each possible brand into a separate 0/1 column. "Is the brand Nike?" "Is the brand Adidas?" "Is the brand New Balance?" Works for small categorical sets but explodes when you have 10,000 brands.
- **Ordinal encoding**: assign each category an integer. Simple but implies a meaningless order (Nike=1, Adidas=2 — does Adidas mean "more" Adidas? No.).
- **Target encoding**: replace each category with the average label value for that category. Nike → "average click-through rate of Nike products." Useful but risks data leakage if not done carefully.
- **Embedding**: train a small lookup table that gives each category its own learned vector. Powerful, but requires training and is overkill for small problems.
- **Match indicators**: instead of encoding the brand, encode "does the query brand match the product brand?" That is a single 0/1 feature. Often more useful for ranking than encoding the brand itself.

DruSearch leans on match indicators and short ordinal codes for things like derived gender (men's=1, women's=2, unisex=3). It avoids high-cardinality one-hot encodings, which would balloon the feature vector.

Feature parity: the highest-risk correctness boundary

A feature has to be computed two places: in the **training pipeline** (where the model learns) and in the **serving system** (where the model is used to score live queries). If the two computations disagree — even slightly — the model becomes silently broken.

Imagine training a model where feature 7 is "log of price in cents," and serving a model where feature 7 is "log of price in dollars." The model expects an input around $\log(2999) \approx 8.0$ and gets $\log(29.99) \approx 3.4$. The score is garbage. Worse, your tests might still pass and your metrics might look only slightly off.

This is called the **feature parity problem**, and it is the single most common silent failure in production ML systems. The defenses include:

1. A **shared schema**: a single source of truth listing the features, their order, their types, and how each is computed. DruSearch uses a JSON file (`feature_schema.json`) that both Python (training) and Go (serving) read.
2. **Code generation**: from the shared schema, generate the Python and Go data structures. The schema is the source of truth; the language-specific code is regenerated when the schema changes.
3. **Cross-language fixtures**: a set of (input, expected output) test cases that both the Python feature builder and the Go feature builder must produce identical results for. Run these in CI ("Continuous Integration": automated tests on every change).
4. **Schema versioning**: the schema has a version number. Models are trained against a specific version; the API refuses to load a model whose feature schema version does not match the one it was built for.

These guards seem heavy for a small system. They are absolutely essential because the failure mode is catastrophic and silent. A model that scores garbage features still produces numbers; the numbers are just wrong, and your search quality quietly tanks.

Chapter 9: Labels — Teaching the Model What “Good” Means

A ranking model needs **examples** to learn from. Each example is a feature vector (Chapter 8) plus a **label** — a number indicating how good that result was. The model learns to predict the label from the features. Then, at query time, we use those predictions to sort candidates.

The label is the model's only source of truth. Get it wrong, and the model learns the wrong thing.

This chapter is about where labels come from, and why getting them right is harder than it looks.

The dream and the reality

The dream: every (query, document) pair has an objective relevance score. A relevance of 4 means “perfect answer.” A 0 means “completely irrelevant.” We collect labels, train, and we are done.

The reality: nobody is sitting there labeling pairs. Even when somebody is, their labels are slow, expensive, and incomplete. So we have to get labels from somewhere else: from user behavior, from auxiliary models, from reusable judgments produced by other organizations.

There are three main label sources, each with strengths and traps.

Source 1: human relevance judgments

The cleanest labels come from humans explicitly grading (query, document) pairs. Some standard datasets in the search world:

- **TREC** (Text REtrieval Conference): annual research evaluations going back to the 1990s, with human-graded relevance judgments for each task.
- **MS MARCO** (Microsoft Machine Reading Comprehension): a large-scale collection of queries paired with passages, with relevance judgments.
- **ESCI**: an Amazon-released dataset for product search. Each (query, product) pair has a label drawn from {E, S, C, I}:
 - **E (Exact)**: this product is exactly what the user wanted.
 - **S (Substitute)**: not exactly what they wanted, but a reasonable alternative.
 - **C (Complement)**: a product that complements but does not substitute (e.g., shoelaces for “running shoes”).
 - **I (Irrelevant)**: not relevant.

These are called **graded relevance labels**: relevance is on an ordered scale, not just yes/no. The grades typically map to numeric labels for training, e.g., E=4, S=3, C=2, I=0. (DruSearch uses these mappings.)

Human-judged labels are the gold standard. Their problem is that they are limited: ESCI has tens of thousands of queries, but a real product catalog has millions of (query, product) combinations and most are unlabeled. You cannot judge them all.

Source 2: behavioral labels (clicks, purchases)

The opposite extreme: derive labels from what users actually do. If a user clicks a result, it must have looked interesting. If they buy it, it must have been good.

Behavioral signals are the lifeblood of large-scale search. They are cheap, plentiful, and constantly updated. But they are also dangerously biased, in ways that the next chapter (Chapter 10) is entirely about. For now, understand that behavioral labels are powerful but tricky.

The basic events typically logged are:

- **Impression:** the user saw a result. This is the *exposure* event — the denominator in any rate calculation.
- **Click:** the user clicked a result. Implies enough interest to investigate further.
- **Purchase** (or **conversion**): the user bought the product, or completed the action the search was meant to enable. Strong positive signal.

A simple behavioral label scheme: purchase = 4, click = 2, no engagement = 0. The numbers are ordinal — purchase is “more positive” than click — but the exact gaps are arbitrary. Different teams use different mappings.

Source 3: model-generated labels (teacher models)

A more recent technique. If you have an *expensive but accurate* model (say, a large neural network that scores (query, document) pairs in 100 milliseconds each), you cannot use it at query time — too slow. But you *can* use it offline to score millions of (query, document) pairs, then use those scores as labels to train a *cheap, fast* model that mimics it.

This is called **knowledge distillation**, and the expensive model is called a **teacher**. The cheap model is called a **student**. We will return to this in Chapter 12.

DruSearch uses all three label sources:

- **ESCI judgments** are the authoritative labels when available.
- **BGE teacher scores** fill in additional labels for unjudged pairs.
- **Behavioral events** feed personalization features (rather than direct labels), to avoid the position bias trap we are about to introduce.

Combining label sources

When you have multiple label sources, you face a question: which one wins?

DruSearch's policy, which is a sensible default:

1. If an ESCI judgment exists for a (query, product) pair, use it. Human labels are the most reliable.
2. Otherwise, optionally use a teacher-model score (with reduced weight, because it is less reliable than a human).
3. Behavioral data does not become a label directly; it becomes a feature.

The reason for option 3 is the topic of Chapter 10.

Sample weights

Even within a single label source, not all examples are equally trustworthy. A product with 10,000 impressions has a more reliable click-through rate than one with 5. A label assigned by a strong teacher model deserves more trust than one from a weak heuristic.

To handle this, training examples carry a **sample weight**. The weight is a multiplier on how much the model “cares” about getting that example right. A weight of 1.0 is the default; a weight of 0.3 means this example matters one-third as much as a normal example. Strong human labels get weight 1.0; teacher-generated labels might get 0.5; weak heuristic labels might get 0.2.

Sample weights let you mix label qualities without throwing away the weaker ones.

Chapter 10: The Click Trap — Position Bias

This chapter is about a single mistake, and it is so common and so consequential that it deserves its own chapter. The mistake is: **using clicks as a direct label for relevance**.

It looks innocent. Users click things they like. So a clicked result is a positive example, an unclicked one is negative, and we can train a ranker on millions of clicks. What could go wrong?

A great deal could go wrong. The mistake has a name — **position bias** — and stepping into it has destroyed more search systems than any other failure mode.

The trap

When users see a search results page, they look at the top result first. They look at the second result second. They almost never look at result 17.

So result 17 has a low click rate not necessarily because it is bad — but because nobody saw it. And result 1 has a high click rate not necessarily because it is good — but because everybody saw it.

If you train a ranker using "got clicked = good, did not get click = bad," the ranker will learn one thing very well: **rank the same things at the top that the previous ranker put at the top**. It will not learn what is actually relevant. It will learn to imitate position.

This is called a **feedback loop**, and it is corrosive. The new ranker does not improve over the old one — it just freezes the old one's biases in place. Over time, anything that was once at the top will stay at the top, regardless of whether it should. New, better products that are not yet popular cannot break in, because they get no clicks, because they are not shown at top.

Worse, the loop is hard to detect. Your offline metrics on click data will look great — the new model predicts clicks brilliantly. Your online metrics will look mediocre. The model is doing exactly what you trained it to do; it just turns out that what you trained it to do was not what you wanted.

Where position bias comes from

A few overlapping causes:

- **Visual attention.** Top results are physically more visible. Users start at the top and scroll down only if dissatisfied.
- **Trust.** Users assume the search engine is doing its job — that the top result is the best. If they see a vaguely plausible match at position 1, they click it without checking position 2.
- **Task abandonment.** Users with limited patience stop after the first few results, even if a better one exists at position 8.

The result: even if you put a *terrible* result at position 1, it will still get more clicks than a *great* result at position 17.

Symptoms of position-bias collapse

If you have trained a ranker on raw click data, you might see:

- Offline metrics look great, online metrics flat or worse.
- The ranker's top results look almost identical to the previous ranker's, no matter what features you add.
- New or rare products never seem to rank well, even when they should.
- Diversity is poor — the top 10 results all look similar.
- Adding more training data does not help.

These are all signs that the model has learned to predict *clicks given position*, not *relevance*.

How to escape the trap

There is no single fix; production systems use multiple defenses, often combined. The most important ones:

1. **Use independent labels.** If your labels come from a source that does not depend on what the previous ranker showed (human judgments, teacher models, exact-match purchases), position bias largely disappears. DruSearch's primary labels are ESCI judgments and BGE teacher scores precisely for this reason.
2. **Inverse propensity scoring (IPS).** A formal correction: estimate the probability that a result was *seen* given its position (its "propensity"), and weight clicks inversely to that probability. A click at position 17 counts much more than a click at position 1, because it took unusual effort. IPS is well-studied and works, but it requires good propensity estimates and careful implementation.
3. **Randomization (interleaving).** Show some users a randomized or shuffled set of results and use those clicks as a less-biased signal. This costs some user experience but produces clean training data. Used by big search engines (Google, Bing) for parts of their training data.
4. **Use behavior as features, not labels.** Aggregated user behavior — "this product gets clicked 5x more than average when shown" — can become a *feature* without becoming a *label*. Other features (true labels, query-doc match scores) bear the burden of telling the model what is relevant; the behavior feature provides a useful signal without dictating the answer. DruSearch uses popularity priors and CTR priors as features for exactly this reason.
5. **Train across positions deliberately.** Even when using behavior as a feature, randomize position during exploration so you collect data at every rank. A small fraction of randomly reordered impressions can save you from runaway feedback loops.

A recurring lesson

Position bias is a specific instance of a more general principle: **whenever your training data is generated by the system you are training, you have a feedback loop, and you must reason carefully about what your data measures versus what you actually want to learn.**

Search rankers generate click data, which becomes training data, which trains the next ranker. Recommendation systems recommend things, which gets watched, which trains future recommendations. Risk-scoring systems route work to humans, whose decisions become future training labels. In all of these, the data is shaped by the system, and a naive "just train on what users did" approach leads the system to recreate itself rather than improve.

This is not a flaw in machine learning. It is a structural property of any closed-loop system. The defense is awareness, plus deliberate decoupling: bring in labels from outside the loop (humans, teachers, randomization, exact-judgment datasets) so the model has something to learn besides its own past.

Chapter 11: Learning to Rank

We are now ready for the centerpiece. We have features (Chapter 8), we have labels (Chapter 9), and we have learned to be careful about where the labels come from (Chapter 10). The remaining question: how do we actually train a model to rank results?

The umbrella term is **Learning to Rank**, abbreviated **LTR**. It refers to a family of machine-learning techniques specifically for the problem of ordering items in response to a query.

This chapter covers the three main LTR formulations, the algorithms that dominate in production, and what training a ranker actually looks like.

Three ways to formulate ranking as ML

Ranking is unusual as a machine-learning problem. Most ML problems are about predicting a single output — the price of a house, the next word in a sentence, whether an image contains a cat. Ranking is about producing an *ordering* over a set, which is a different shape.

There are three standard formulations:

Pointwise

Treat each (query, document) pair independently. Train a model to predict the relevance label for that pair. At serving time, score each candidate, sort by score.

This is just **regression** (predicting a number) or **classification** (predicting a category). It is the simplest formulation, and you can use any standard regression or classification algorithm: linear regression, gradient-boosted trees, neural networks.

The drawback is that the model never sees the *comparison*. It does not know that document A and document B are competing for the same query. It only sees one (query, document) at a time. As a result, pointwise ranking often misses the relative-quality signal that is the whole point of ranking.

Pairwise

Train a model to predict, for any two documents under the same query, which is more relevant. The training data is pairs: (query, doc_A, doc_B, A_is_better).

This forces the model to learn relative differences. **RankNet**, **RankBoost**, and the famous **LambdaRank** (the foundation of LightGBM's ranking implementation) are all pairwise methods.

The drawback: pairwise methods optimize for *pair correctness*, not for *list quality*. Getting one pair wrong at positions 1 and 2 is much worse than getting one pair wrong at positions 99 and 100 — but a pure pairwise method treats them equally.

Listwise

Train a model to optimize a quality metric over the entire result list directly. The training signal is the full list ordering and a metric like NDCG (defined below).

Listwise methods are mathematically harder because metrics like NDCG are non-differentiable (small changes in score can leave the ordering unchanged, then suddenly flip it). Tricks like **LambdaMART** approximate listwise optimization by reweighting pairwise comparisons by their effect on the list metric.

In practice, modern LTR is dominated by **listwise-flavored pairwise methods**: pairwise comparisons with weights chosen to approximately optimize a list-level metric. **LambdaMART** and **LambdaRank** are the canonical examples. Both **LightGBM** (DruSearch's default) and **XGBoost** (its alternative) implement them.

Gradient-boosted decision trees

The dominant algorithm family for production LTR is **gradient-boosted decision trees** (often abbreviated **GBDT** or **GBT**). The two big libraries:

- **LightGBM**: developed by Microsoft, very fast, the de-facto default for ranking tasks.
- **XGBoost**: developed at the University of Washington and now widely used; slightly different optimizations, sometimes more accurate, sometimes slower.

A **decision tree** is a model that asks a sequence of yes/no questions about the input and follows the branches to a leaf, which contains a predicted score:

```
Is BM25 score > 5.0?
  Yes -> Is brand match = 1?
          Yes -> 4.2
          No  -> 1.7
  No  -> Is RRF score > 0.05?
          Yes -> 0.8
          No  -> 0.1
```

A single decision tree is weak. **Boosting** is the trick of training many trees in sequence, each one trained to correct the residual errors of the previous. After a few hundred trees, the combined model is very strong.

Why are GBDTs so popular for ranking?

- They handle mixed numeric and categorical features naturally.
- They do not require feature scaling — the model is invariant to monotonic transformations.
- They train quickly even on millions of examples.
- They are interpretable. You can ask "which features mattered most?" and get a meaningful answer.
- For tabular data — like our 29-feature ranking vectors — they are usually as good as or better than neural networks.

Neural networks are dominant in vision, language, and audio. For tabular ranking with hand-engineered features, GBDTs still win. This is part of why the search-engineering job has not been entirely absorbed into deep-learning.

What training looks like

The training process for an LTR model:

1. **Assemble training data.** Each row is one (query, document, user) triple, encoded as a feature vector and labeled with a relevance score and a query identifier. Rows from the same query are grouped together — the model needs to know "these candidates all came from the same query" to compare them properly.

2. **Split into train / validation / test.** **Train:** the model sees this data and adjusts its parameters to fit it. **Validation:** used during training to detect overfitting and choose hyperparameters (how many trees, how deep, etc.). **Test:** held out entirely; used only at the end to estimate true performance.
3. **Choose the objective.** For LightGBM ranking, the objective is `lambdarank` with `label_gain=[0, 1, 3, 7, 15]` (a mapping from integer label to "gain," weighting higher labels more aggressively). For XGBoost, it is `rank:ndcg`.
4. **Train.** The library iterates: build a tree, evaluate on validation, build another tree to fix the previous tree's errors, and so on. Stop when validation performance plateaus.
5. **Evaluate.** On the held-out test set, compute ranking metrics (next chapter).

The output is a **model artifact**: a file containing all the trees and their structure. For LightGBM it is typically a text file, a few hundred KB to a few MB. For XGBoost it is JSON. This artifact is what gets shipped to production.

Splits matter more than you think

The split between train, validation, and test is more delicate in ranking than in most ML problems. The reason is **query overlap**: if the same query appears in train and test, you are not measuring whether the model generalizes to new queries — only whether it generalizes to new documents under known queries.

Best practice: split *by query*, so that any query in the test set is unseen in training. ESCI provides canonical query splits for exactly this reason. DruSearch follows them.

Within the training set, a further train/validation split is also typically done by query — using a stable hash of the normalized query string to assign each query to train or validation. The "stable hash" matters because you want repeatable splits across runs, not random ones that change every time.

Avoiding overfitting

Overfitting is the failure mode where a model performs well on training data and poorly on new data. It happens when the model memorizes the training examples instead of learning the underlying pattern.

For GBDT rankers, the standard defenses:

- **Early stopping:** track validation performance during training; stop adding trees when validation stops improving.
- **Limit tree depth:** deeper trees are more expressive but more prone to overfitting.
- **Regularization:** penalize complex models by adding a small term to the loss for each leaf or each non-zero weight.
- **More data:** nothing fights overfitting like more diverse training examples.

In a ranking system with 29 features and tens of thousands of queries, overfitting is a real risk. It is detected in training (validation curves diverging from training curves) and confirmed in evaluation (test set performance drops below validation).

Chapter 12: Teacher Models and Distillation

Earlier we mentioned three label sources: human judgments, behavioral data, and model-generated labels. The third deserves its own chapter, because it is the most subtle and one of the most useful techniques in the modern search toolbox.

The motivation

Suppose you have a small ranker — a few hundred trees, scoring a candidate in 3 milliseconds. It is fast enough to run at query time. Its quality is "okay." You want it to be much better.

You also know about a different kind of model: a large **cross-encoder** that takes a (query, document) pair as input and produces a relevance score. Cross-encoders are very accurate. They genuinely understand the relationship between the two pieces of text. The problem: they take 50 to 500 milliseconds *per pair*. To rerank a hundred candidates with one, you would need 5 to 50 seconds. Unusable at query time.

So the cross-encoder is too slow to serve. The fast ranker is fast enough but not as accurate. Can we get the best of both?

Distillation

The idea, called **knowledge distillation**: use the slow, accurate model offline to score huge numbers of (query, document) pairs. Use those scores as labels to train the fast model. The fast model is being trained to *imitate* the slow one.

The terminology:

- **Teacher**: the slow, accurate model. Produces target scores.
- **Student**: the fast, simpler model. Trained to predict the teacher's scores.

The student does not become as accurate as the teacher. But it gets considerably closer than it would have without the teacher's signal — and it runs in milliseconds.

Why does this work? Several reasons:

1. The teacher provides a much denser signal than human labels do. You can teacher-score millions of pairs; you cannot human-judge millions of pairs.
2. The teacher's scores capture nuance that binary labels miss. A teacher score of 0.92 is meaningfully different from 0.78, even though both might map to the same human "relevant" label.
3. The student does not need to *be* the teacher; it just needs to *approximate* it well in the cases that come up at query time. With enough training data, that approximation is good.

The BGE teacher

DruSearch's teacher is a model called **BGE-reranker-v2-m3**. It is a cross-encoder: takes (query, product) as input, returns a relevance score. It is dramatically more accurate at judging text relevance than BM25 or even most dual-encoder embedding models, because it actually attends to the interaction between query and document tokens — the BM25 score sees only word overlap; the BGE reranker sees full meaning. But it is slow.

In DruSearch, the BGE teacher runs offline as part of the training pipeline. It scores every training candidate that does not already have an authoritative ESCI judgment. Those scores become weak labels with reduced sample weight, mixed in with the strong ESCI labels.

The offline cost is real — scoring tens of thousands of pairs can take an hour on a laptop. But it happens infrequently (only when retraining the model, not at every search), and on a developer's machine using GPU acceleration when available, it is tractable.

The student is the LightGBM ranker. It runs in 3 milliseconds at query time and produces decent rankings even though it is much simpler than the BGE teacher.

When (and when not) to use a teacher

Distillation is a good fit when:

- You have a strong model that is too slow to serve.
- You can run the strong model offline at scale.
- Your fast model has the *capacity* to learn from the strong model's signal.

It is a bad fit when:

- The strong model is also wrong. The student inherits the teacher's biases.
- The teacher is biased toward the wrong objective (for instance, a question-answering model used as a relevance model when "relevant for shopping" is what you actually want).
- The downstream features are insufficient. If your fast model has 5 hand-coded features and the teacher's nuance lives in subtleties those features cannot represent, distillation has a ceiling.

The serving-vs-training divide

Notice the architectural pattern that emerges: the slow, expensive model lives **offline**; the fast model lives **online**. The teacher does not exist in production at all. There is no service in DruSearch's API path that calls the BGE reranker — the API does not even know it exists. The teacher's role is to train the student. After training, the teacher goes back to sleep until the next training run.

This **online/offline divide** is a recurring pattern in production ML systems. Heavy computation, complex models, and exhaustive scoring all happen offline. The serving path is kept lean. Chapter 16 develops this pattern further.

Part IV — Behavior and Personalization

Chapter 13: Events — The Feedback Loop

A search system that does not learn from its users is a search system that gets worse over time, because the world changes around it. New products, new user preferences, new vocabulary, new seasonal trends — none of these can be picked up without observing what users actually do.

That observation requires events.

What an event is

An **event** is a logged record of a user action. For search, the standard events are:

- **Impression**: the user saw a particular result on a search results page. Recorded with the query, the user (if known), the session, the rank position, and the displayed product.
- **Click**: the user clicked a particular result.
- **Purchase** (or, more generally, **conversion**): the user took the action that the search was meant to enable. For e-commerce: bought the product. For news: read the article to the bottom. For documentation: stayed on the page for more than 30 seconds without bouncing.

The choice of which events to log depends on what the search system is for. The choice of *which* event signals success matters more than people usually think. A click is not a success: it might lead to disappointment, an immediate back-button, and a refined query. A purchase is closer to a success but not perfect — the user might return the product. A "no follow-up search within 60 seconds" is sometimes used as a proxy for satisfaction.

For our purposes, the typical hierarchy is: impression < click < purchase, with each being a stronger positive signal than the last.

The event log

Events are stored append-only in a transactional store. DruSearch uses Postgres for this; large-scale systems often use Kafka, Kinesis, or similar streaming platforms. Append-only matters because:

- You never modify or delete past events. They are historical fact.
- Many things downstream consume events: training pipelines, analytics dashboards, fraud detection, A/B test analysis. Each may need to replay events from the beginning.
- The ordering of events sometimes matters (sessions, conversion attribution).

The event log is the **system of record** for user behavior. Almost everything in Part IV is built from it.

Event quality

A subtle issue: events you log are exactly the events your downstream pipelines have. If you forget to log impressions, you cannot compute click-through rates. If you log clicks but not the rank position they appeared at, you cannot correct for position bias. If you log without timestamps, you cannot measure session activity.

The boring rule: log everything you might possibly need, with all the context (query, user, session, rank, score, model version) you might want. Storage is cheap; missing data is permanent.

Sessions

Most queries do not happen in isolation. A user types one query, looks, refines, types another, clicks, and buys. The whole sequence is a **session**. Sessions are valuable because:

- The fact that a user *refined* a query (typed a new one within seconds) is a signal that the previous query failed to satisfy them.
- Time-on-page after a click is a signal of relevance.
- A click followed by an immediate back-button is a "pogo stick," a strong negative signal.

DruSearch attaches a `session_id` to every search request and event. The session is a lightweight identifier (created on the first request, propagated through cookies or returned to the client) that ties together a user's recent activity. Sessions typically time out after 30 minutes of inactivity.

You do not need to do anything sophisticated with sessions to get value from them — even just having the IDs lets you join events into sequences for analysis. More sophisticated uses (session-aware ranking, query-rewriting based on past queries in the session) come later.

Async event ingestion

Logging events on the search hot path is dangerous: if the event log is slow or unavailable, do you delay returning results? Almost certainly not. The user does not care about your event log; they care about getting results.

The standard pattern: **asynchronous event writing**. The search handler returns the response immediately. Events are queued in memory and written to the event store by a background worker. A short queue means almost no events are lost in a crash; an unbounded queue would eat memory if the writes back up; so the queue is bounded with a sensible policy on overflow (drop oldest, drop newest, log a metric).

DruSearch's API has an `eventbus` package that does exactly this: a goroutine drains an in-memory queue into Postgres in batches. Search performance is unaffected by Postgres availability for event writes.

What events become

Events feed into many things:

- **Aggregated product features:** how often was this product impressed, clicked, purchased, in the last N days? These become the popularity priors and CTR priors discussed in Chapter 8.
- **Aggregated user features:** which brands has this user clicked or bought? Their preferences. Become the personalization features in Chapter 14.
- **Training data:** pairs of (query, product) that became candidates and either did or did not get engagement. Used as features (with the position-bias caveats from Chapter 10).
- **Online metrics:** click-through rate, conversion rate, average rank of clicked results. Used to monitor the system.
- **A/B test analysis:** comparing two ranker variants by their downstream conversion rates.
- **Anomaly detection:** a sudden drop in click-through rate is a signal that something broke.

The whole system is, in a sense, a giant cycle: queries produce results, results produce events, events produce features and labels, features and labels produce a new ranker, and the new ranker produces better results. The feedback chapters (10) warn about how this cycle can go wrong; the rest of this part is about making it go right.

Chapter 14: Personalization

If two users type the exact same query, should they get the exact same results?

Not necessarily. A user who has bought from a particular brand five times is more likely to want that brand again. A user who consistently buys the cheapest item probably wants the cheapest item. A user who has never visited the site has no history but should still get a reasonable default.

Personalization is the practice of adjusting search and ranking based on the individual user. It is a powerful lever for quality, but it is also the area of search engineering most likely to go wrong silently. This chapter is about both.

Levels of personalization

Personalization is a spectrum, not a switch. Several common levels:

1. **None.** Every user gets the same results for the same query. Simplest and most predictable.
2. **Cohort-based.** Users are grouped (by location, language, device, demographic), and each cohort gets its own ranking adjustments. Limited but reliable.
3. **Aggregated user features.** Per-user signals, like "average price the user has bought" or "fraction of purchases from each brand," are used as inputs to the ranker. The ranker decides how much they matter for each query.
4. **Per-user models.** A separate model (or model fragment) for each user. Almost never worth it — too much complexity, too little data per user, and users want to be surprised sometimes.
5. **Sequence-aware models.** Models that consume the user's recent activity stream as a sequence and produce context-aware predictions. State of the art for some recommender systems; overkill for most search.

DruSearch sits firmly at level 3: a small set of aggregated user features feeds into the ranker. Specifically, it tracks each user's brand affinity — the share of clicks each brand has received from this user historically — and exposes one feature, `user_brand_affinity`, to the ranker for each candidate.

That single feature is enough to get most of the personalization benefit. The ranker learns: "if this user's affinity for the candidate brand is 0.4 (40% of their clicks), boost it; if it is 0.0, ignore."

The cold-start problem

What about a brand-new user with no history? Their personalization features are empty. The system has to fall back to non-personalized ranking.

This is called the **cold-start problem**, and it shows up in every recommendation and search system. Several common defenses:

- **Reasonable defaults.** Empty features map to neutral values (zero, average), so the ranker still produces something sensible.
- **Cohort fallback.** Use coarser signals (location, time of day) when individual signals are missing.

- **Train with masking.** Show the model that some training examples have empty user features, so it learns to handle them gracefully. DruSearch trains with 30% anonymous masking — 30% of training rows have their user features cleared out, so the model is forced to be good without them.

Anonymous masking is worth its weight in gold. Without it, the ranker learns to depend heavily on personalization features and crashes on anonymous traffic. With it, the ranker degrades gracefully.

Online vs. offline user features

User features come in two flavors:

- **Slow features:** aggregated over weeks or months. Brand affinity, average cart size, preferred categories. These change slowly and can be precomputed in batch jobs (overnight, hourly).
- **Fast features:** about the current session. Did the user just search for something else? Did they look at a product detail page in the last 30 seconds? These need to be updated in seconds and consumed at query time.

Slow features live in a feature store — DruSearch uses Redis, which is a fast in-memory key-value store, with each user's feature hash at a key like `feat:user:{user_id}`. At query time, the API pulls the hash for the current user and uses it to populate features.

Fast features live in a session cache or in the request itself, attached as the user navigates. DruSearch does not currently use fast features, but the architecture has space for them.

The privacy and fairness dimension

Personalization based on per-user behavior raises real concerns that are not optional:

- **Privacy.** User data has to be stored, transmitted, and deleted properly. Regulations like GDPR (the EU's General Data Protection Regulation) and CCPA (the California Consumer Privacy Act) impose specific requirements: users must be able to view, export, and delete their data; sensitive categories require extra care; data retention has limits. None of this is technically hard, but it is legally non-optional and easy to overlook in early development.
- **Fairness.** Per-user personalization can amplify existing patterns in problematic ways. A user who has only clicked on cheap, low-quality products may get those exclusively, even when they are searching for something better. A user from a particular demographic group may receive systematically different prices or recommendations. These outcomes are sometimes unintentional emergent effects of optimizing for click-through rate.

A search system without personalization can ignore these; one with it cannot. The standard mitigations include limiting the strength of personalization (don't let it dominate other signals), regular fairness audits, careful logging and access control on user data, and giving users control (the ability to clear their history).

The "filter bubble" pitfall

One specific failure mode of aggressive personalization. If the ranker only ever shows the user things very similar to what they have already liked, they never discover anything new. Over time, their session looks like a hall of mirrors. This is sometimes called a **filter bubble**.

The defense: **explore vs. exploit**. Most of the time, show the user what the ranker thinks is best (exploit). Some fraction of the time, show something the ranker is uncertain about (explore). The exploration produces data that improves the ranker and breaks the user out of bubbles. Tuning the exploration fraction is a real engineering choice.

DruSearch's current personalization is mild enough (one feature out of 29) that filter bubbles are not a serious risk. As personalization gets stronger, the explore-exploit question becomes important.



Part V — Evaluation and Operations

Chapter 15: Did It Get Better?

Offline Evaluation

You changed something — added a feature, retrained the ranker, swapped models. Is the system better than before? This is the question that **evaluation** answers, and getting it right is harder than it looks.

Two kinds of evaluation

There are two complementary approaches:

- **Offline evaluation.** Use historical data and known labels to compute ranking metrics on a held-out set. Cheap, fast, repeatable. Done before deploying.
- **Online evaluation.** Deploy the change to a slice of real traffic and measure user behavior. Slow, expensive, but the ground truth.

This chapter covers offline evaluation. Online evaluation (A/B testing) is mentioned briefly later but is mostly out of scope for this book.

Ranking metrics

A handful of standard metrics define what "good ranking" means.

Recall@k

Of all the relevant items, what fraction did we surface in the top k results?

`recall@k = (relevant items in top k) / (total relevant items)`

Useful for retrieval — "did we even find the good stuff?" Less useful for the final ranker, because it does not care about the order within the top k.

Precision@k

Of the top k results, what fraction are relevant?

`precision@k = (relevant items in top k) / k`

Useful when "relevant" is binary. For graded relevance (the ESCI E/S/C/I scale), it loses information.

MRR — Mean Reciprocal Rank

For each query, find the rank of the first relevant result. Take the reciprocal (1/rank). Average across queries.

`MRR = average over queries of (1 / rank of first relevant result)`

A first-relevant-result at rank 1 contributes 1.0; at rank 2 contributes 0.5; at rank 10 contributes 0.1. MRR is good for "find me the answer" tasks like web search where users typically only want one good result.

NDCG — Normalized Discounted Cumulative Gain

The most important metric for graded-relevance ranking.

It is built up from three pieces:

1. **Gain:** a numeric "value" for a result based on its label. $E=4$ contributes more gain than $S=3$ contributes more than $C=2$.
2. **Discount:** results lower on the page contribute less. The standard discount is $1/\log_2(\text{rank}+1)$, so rank 1 is undiscounted, rank 2 is divided by $\log_2(3) \approx 1.58$, rank 3 by $\log_2(4) = 2$, and so on.
3. **Normalization:** divide by the best possible score for that query (the **ideal** ranking — the one where every relevant item is in its perfect position).

The result is a number between 0 and 1 for each query, then averaged across queries:

$\text{DCG@k} = \text{sum over top-k results of: gain(label) / discount(rank)}$

$\text{IDCG@k} = \text{the same, but for the ideal ranking}$

$\text{NDCG@k} = \text{DCG@k} / \text{IDCG@k}$

Why NDCG is so widely used:

- It handles graded relevance properly.
- It rewards getting the very top results right (because of the discount).
- Normalization makes scores comparable across queries with different numbers of relevant items.

DruSearch's primary offline metric is **NDCG@10** — NDCG over the top 10 results. The cutoff at 10 reflects the fact that real users rarely look beyond the first page.

Lift

When comparing a new ranker to a baseline, the **lift** is the relative improvement:

$\text{lift} = (\text{new_metric} - \text{baseline_metric}) / \text{baseline_metric}$

A lift of 0.05 means a 5% relative improvement. Lift is what teams ship on. A 1% lift in NDCG@10 is a real, defensible improvement; under that, it is in the noise of natural variation.

Building an offline eval set

The standard structure:

1. **Pick a set of queries.** Ideally a representative sample from real traffic, not just easy cases.
2. **For each query, identify all known-relevant documents.** This is where labels come in — ESCI judgments, human labels, teacher scores.
3. **For each query, run the ranker.** Record the order of the top results.
4. **Compute NDCG@10 (or whichever metric) per query, then average.**

Two pitfalls to avoid:

- **Train/test leakage.** Make sure the queries used for evaluation are not also in the training set. Otherwise, the model has memorized them and the metric is meaningless.
- **Coverage limits.** If you only judged 10% of (query, document) pairs, an unjudged document at rank 1 looks "irrelevant" in the metric, even if it is actually a perfect result. NDCG is sensitive to this; it can punish a good model for surfacing items the human judges never saw. The defense: be aware of judgment coverage, prefer datasets with high coverage (ESCI is decent on this), and supplement with judgments-on-demand or teacher scores for unjudged items.

RRF as a baseline

A useful sanity check: compute NDCG for the **RRF baseline** — that is, the ordering produced by hybrid retrieval before the LTR reranker runs. If the LTR reranker's NDCG is *lower* than RRF's, your reranker is making things worse, and you should not promote it. This is not hypothetical; it happens regularly when models are trained on bad labels (Chapter 10) or when feature parity breaks (Chapter 8).

DruSearch's training pipeline always logs both: the RRF baseline NDCG and the LTR NDCG. The lift between them is the headline number. A lift below zero blocks promotion.

Promotion gates

The combination of "compute metrics on test data + refuse to deploy if they are not better" is sometimes called a **promotion gate**. It is a small piece of automation that pays for itself many times over. A typical gate:

- Train a new candidate model.
- Evaluate it on the held-out test set.
- Compare NDCG@10 against the currently-served model (or a fixed historical baseline).
- If the new model is strictly better (or at least non-regressing), allow promotion.
- If not, log a warning and refuse.

This catches bad training runs, broken features, accidental label flips, and many other surprises before they reach users.

Chapter 16: The Two Worlds — Online and Offline

A recurring pattern has emerged across multiple chapters. The teacher model lives offline, the student model lives online. Slow user-feature aggregations live offline; fast feature lookups live online. Training is offline; serving is online. It is worth pausing to make this divide explicit, because it organizes nearly the entire architecture of a production search system.

The two worlds

Online is the request path. The user is waiting. Latency is measured in milliseconds. Failures must be graceful. Resources are limited because everything in this path competes for the same fast machines. Code in the online path needs to be carefully optimized, and changes to it deploy carefully.

Offline is everything else. Batch jobs that run nightly. Pipelines that train models. Aggregations that recompute features. Evaluations that compute metrics. There is no user waiting; failures can be retried; computation can be heavy because it runs on different infrastructure.

Most production search systems have a sharp dividing line between the two. Things that *can* be moved offline *should* be moved offline. The online path is precious and must be protected.

Examples of the divide

Concern	Online	Offline
Embedding the query	Yes — needed per request	No
Embedding products	Cached online — but the <i>computation</i> happens offline at ingest time	Yes — at ingest time
BM25 retrieval	Yes	No
Vector retrieval	Yes	No
RRF fusion	Yes	No
Ranker scoring	Yes — but the model is small and fast	No
Computing features	Mostly online — but priors are precomputed offline	Pre-aggregations and per-user
Training the ranker	No	Yes
Teacher model scoring	No — too slow	Yes — used for label genera
Evaluating the model	No	Yes
Promoting the model	No	Yes — promotion writes a r

Feature stores

The connection between the two worlds is the **feature store**: a fast lookup service where offline-computed features are written, and online services read from at request time.

For DruSearch:

- Per-user aggregations (brand affinity) are computed offline by the `features.user_aggs` pipeline and written to Redis under `feat:user:{user_id}`.
- Per-product aggregations (popularity, CTR priors) are computed offline by `features.aggregates` and written to a Postgres table `product_features`.
- At request time, the API reads both: Redis for the user, Postgres for each candidate product.

The feature store pattern is universal in production ML. Some systems use specialized feature-store products (Feast, Tecton, Hopsworks); others, like DruSearch, just use the existing transactional and cache stores. The principle is the same: precompute heavy aggregations offline, serve them quickly online.

The training/serving skew problem

A persistent challenge: the offline pipeline that computes features for *training* and the online code that computes them for *serving* must produce identical values. This is the feature parity problem from Chapter 8, and it shows up here too.

Examples of skew:

- Different default values for missing data ("0" online, "null" in training).
- Different time zones in a date-derived feature.
- Different rounding behavior in floating-point math between Python (used for training) and Go (used for serving).
- Code drift: someone changes the online code without updating the training code, or vice versa.

The defense: a single source of truth (the schema), strict version checking at model load, and automated parity tests (the same input must produce the same output in both languages).

What this means for engineering practice

Some practical takeaways:

1. **Default to offline.** Whenever something can be precomputed, precompute it. Never compute at query time what you could have computed last night.
2. **Cache aggressively but carefully.** Caches that go stale silently are dangerous. Invalidation rules need to be explicit.
3. **Treat the model as a versioned artifact.** It is not "the latest code." It is a specific file, with a known version, that can be rolled back.
4. **Protect the online path.** Any new feature, any new computation, gets evaluated for whether it must be online. Most of the time, it must not.

DruSearch is small enough that the divide is manageable in one engineer's head. At scale, this divide becomes the organizing principle of entire teams: an "online serving" team, an "offline ML" team, an "infra" team that maintains the feature store. The principles are the same; the operational complexity scales.

Chapter 17: Latency, Failure, and Graceful Degradation

A search system is a real-time service. If it returns a perfect answer in three seconds, it has failed. Users abandon slow pages, retry their queries, and leave for competitors. This chapter is about the engineering of the hot path: how to make it fast, how to keep it fast, and what to do when something breaks.

The latency budget

The first tool: a **latency budget**. Decide what your maximum acceptable end-to-end latency is, then break it down across components.

DruSearch's budget for a search request:

Stage	p50 target	p99 target
Embedder sidecar	6 ms	15 ms
OpenSearch BM25 + kNN	8 ms	25 ms
RRF fusion	1 ms	3 ms
Redis user features	1 ms	3 ms
LightGBM rerank	3 ms	8 ms
Serialize / HTTP	2 ms	5 ms
End-to-end	~25 ms	<70 ms

A note on the notation: **p50** means "50th percentile" — the median, the latency that half of requests beat. **p99** means "99th percentile" — only 1% of requests are slower. Production systems care a lot about high percentiles, because the slow requests are the ones that wreck user experience and pile up in queues. You will also see **p95** (typical for SLOs) and **p99.9** (for very latency-sensitive systems).

A budget like this serves two purposes:

1. It tells you what to optimize. If RRF fusion is at 5 ms p50 (over its 1 ms budget), you know where to look.
2. It tells you what is acceptable. If the embedder is at 7 ms but you spent two days trying to shave it, stop. It is in budget.

Parallelism and pipelining

Several DruSearch operations run in parallel:

- The query embedding, the BM25 query, and the kNN query *could* all run in parallel (in practice DruSearch fires the embedder first, because both lexical and vector retrieval can wait briefly).
- The Redis user-feature lookup runs in parallel with retrieval.

A naive sequential implementation would be the sum of all these latencies. A parallel implementation is the maximum of the parallel branches plus the sequential ones. The shape of your latency budget directly drives whether you need to parallelize.

A subtler form: **request-time pipelining**. Begin computing things you will need before you strictly know you need them. Speculative work like this can cut latency at the cost of some wasted computation when the speculation is wrong. Search rarely needs to go this far; rendering and other interactive systems sometimes do.

Timeouts

Every external call needs a timeout. Without one, a slow downstream service can hang your hot path indefinitely, exhausting connection pools and cascading the failure into total unavailability.

Setting timeouts well is its own art:

- The timeout should be slightly longer than the realistic worst-case latency. Too short, and you fail under normal load. Too long, and you do not actually protect against slowness.
- Different operations need different timeouts. The embedder might need 100 ms; OpenSearch might need 200 ms; Redis might need 20 ms.
- A timed-out call should fail fast and be handled, not bubble up as an unhandled exception.

Circuit breakers

Worse than slowness is *broken* slowness — a service that is failing every request, slowly. If your code keeps trying to call it, you compound the problem. The pattern that handles this is the **circuit breaker**:

- Track recent failures from a downstream service.
- If failures cross a threshold, "open" the breaker: stop sending requests for a cooldown period, returning a fallback or error immediately.
- After the cooldown, send a few probing requests. If they succeed, "close" the breaker and resume normal operation.

DruSearch wraps the embedder sidecar in a circuit breaker. If the embedder is down or slow, the API stops calling it, falls back to BM25-only mode, and tries the embedder again periodically. The user gets degraded but functional results.

Graceful degradation

The deeper principle: **a search system should degrade gracefully, not catastrophically**.

For DruSearch, the degradation ladder looks like:

Failure	Fallback
Embedder fails or circuit opens	BM25-only retrieval; LTR may still run if features are available, producing <code>bm25+ltr</code> mode
BM25 fails	HTTP 500 (this is the only catastrophic case)
Redis user features fail	Use empty features; treat as anonymous user
LightGBM model unavailable	Return retrieval-order results (no rerank)
ESCI judgments stale	Training degrades quietly; serving unaffected

Notice the asymmetry: most failures degrade to "less personalized, less semantically rich, but still usable" rather than "no response." The exception is BM25 itself — if your inverted index is unavailable, there is no search to do.

Designing for graceful degradation requires deliberate thought up front. The default behavior of most code is to throw on any unexpected condition; making it fall back gracefully means writing fallback paths, testing them, and making sure they actually trigger when expected.

Caching

Caching reduces both latency and load. Common caches in a search system:

- **Query result cache.** "I just answered this exact query a moment ago; here is the answer." Effective for repeated queries (especially common queries on the home page).
- **Embedding cache.** The same query string always produces the same vector; cache it and skip the embedder call.
- **Feature cache.** Per-user features that change slowly can be cached briefly between requests.
- **Document cache.** Rarely useful for search itself; common for product detail pages.

DruSearch is small enough that it does not yet aggressively cache, but the architecture supports adding caches at multiple points. As scale grows, caching moves from optional to load-bearing.

A cardinal rule: every cache must have an invalidation policy. A cache without invalidation is a delayed bug.

Load and capacity planning

Beyond per-request latency, a search system has to handle a certain rate of requests. **QPS** stands for **queries per second**. DruSearch's current target is under 50 QPS — single-machine territory. Bigger systems run at thousands or hundreds of thousands of QPS.

The components have very different scaling properties:

- **OpenSearch / Elasticsearch:** typically scales horizontally by sharding the index across nodes. Read traffic scales nearly linearly with replicas.
- **Embedder sidecar:** stateless, can be replicated trivially. Throughput per replica depends on whether GPUs are available.
- **Redis:** scales with sharding, but key access patterns matter; hot keys can hot-spot.
- **Postgres:** harder to scale for write-heavy workloads. Event ingestion is the most likely bottleneck.

Capacity planning is not in this book's scope, but the headline rule: measure under realistic load, find the first thing that breaks, fix it, repeat. Most search systems hit Postgres or Redis bottlenecks long before they hit retrieval ones.

Chapter 18: Promotion — From Trained to Served

We have a trained model. The offline metrics look better than the previous version. Now what?

The answer is not "git push." A trained model is an artifact, not source code, and shipping it requires a sequence of careful steps designed to prevent the catastrophic failures we have warned about throughout this book. This chapter covers the **model lifecycle**: registry, gates, promotion, hot reloading, and rollback.

Models are versioned data

A common mental mistake is to think of a model the way you think of code. With code, you change something, push, deploy. The new code is the only code; the old code is gone (except in version control).

A model is more like a database row. Old versions still exist. They might be in production. They might be running for some users while a new version runs for others (in an A/B test). When something goes wrong, the right move is often to roll back to a known-good model — not to forward-fix.

This means models need:

- **Identity**: a unique version (a name + tag, a number, a hash).
- **Metadata**: when it was trained, on what data, with what features, what its evaluation metrics were.
- **Storage**: a place where artifacts can be looked up by version.
- **Lineage**: links to the training data and code that produced it.

The standard tool for this is a **model registry**.

MLflow

DruSearch uses **MLflow**, an open-source platform for managing the ML lifecycle. MLflow provides:

- **Experiment tracking**: when you run a training job, MLflow records the inputs (hyperparameters, dataset version, code commit), the outputs (the model artifact), and the metrics (NDCG, lift, training time). All visible in a UI.
- **Model registry**: trained models can be promoted to a "staging" or "production" stage with explicit transitions, version tracking, and rollback.
- **Artifact storage**: where the actual model files live. DruSearch backs MLflow with **MinIO**, a local S3-compatible storage system. In production environments this is usually real S3.

The pattern, with or without MLflow specifically, is universal: training jobs produce versioned, tracked artifacts. Serving jobs consume specific versions. The two are decoupled.

The promotion gate

Before a new model serves traffic, it should pass an automated check. We sketched this in Chapter 15; here is the operational version.

DruSearch's gate (`register.gate`) runs before any model promotion:

1. Load the candidate model.
2. Run it on the held-out evaluation queries.
3. Compute NDCG@10 against ESCI judgments.
4. Compute the same metric for an RRF baseline.
5. Compute the same metric for the currently-served model.
6. Require the candidate to beat the RRF baseline by at least some margin.
7. Optionally require the candidate to beat or match the currently-served model.
8. If all checks pass, allow promotion. Otherwise, fail loudly.

Gates exist to catch:

- A training run that crashed midway and produced a partial model.
- A label rule change that flipped polarity.
- A feature schema mismatch between training and serving.
- Random hyperparameter choices that happened to overfit.
- Bugs in pipeline code that degraded labels.

None of these are theoretical. Each one *will* happen at some point. The gate is what catches them.

Promotion mechanics

Promotion in DruSearch:

1. The trained model is logged to MLflow as an experiment artifact.
2. `register.promote` downloads the artifact, rewrites it if needed for the serving format (LightGBM has a couple of metadata quirks the Go `leaves` library is picky about), and writes it to `models/ltr_reranker.txt` (for LightGBM) or `.xgb.json` (for XGBoost), along with a JSON metadata file.
3. The Go API exposes a `POST /admin/reload-model` endpoint. Hitting it causes the API to re-read the model file from disk and swap its in-memory model handle.
4. The next request uses the new model.

Notable design choices:

- **Filesystem as the interface.** The Go API does not call MLflow; it reads from the filesystem. The promotion pipeline is responsible for putting the right artifact there. This decouples serving from MLflow's availability.
- **Hot reload.** No restart needed. The API atomically swaps the model handle. In-flight requests using the old model finish with the old model; new requests use the new one.
- **Backend-agnostic metadata.** The metadata file declares whether the artifact is LightGBM or XGBoost. The API loads the matching scorer. This lets the same serving code support both backends interchangeably.

Rollback

Hot-reload makes rollback the same operation in reverse: keep the old artifact around, copy it back, hit reload. DruSearch typically commits the served artifact to Git so that any machine with the repo can reproduce a known-good ranker without retraining.

In larger systems, rollback strategies include:

- Keeping the last N model versions in production storage at all times.
- Time-based auto-rollback: if a new model causes a metric to degrade by X% over Y minutes, automatically revert.
- Stage-based deploys: promote to canary → 10% → 50% → 100%, with monitoring at each stage.

These all build on the same primitive: models are versioned, promotions are reversible operations, and the serving system can switch between versions without downtime.

A/B testing

Briefly, because it is mostly out of scope for this book. **A/B testing** is the practice of running two or more variants of a system simultaneously, randomly assigning users to one variant or another, and measuring the difference in outcomes (clicks, conversions, time on site). It is the gold standard for proving that a change is actually better — offline metrics can lie, online metrics with random assignment cannot (much).

DruSearch does not do A/B testing because it is a single-machine demo. In production, every model promotion of consequence should go through an A/B test before serving 100% of traffic. The infrastructure for this is not trivial — it requires consistent assignment (the same user gets the same variant), reliable event logging, and statistical analysis — but it is essential at scale.

Chapter 19: Observability

The last operational topic: knowing what your system is doing in production. This is called **observability**, and it is the difference between a system you maintain and one that surprises you.

The three pillars

Observability is conventionally broken into three categories:

1. **Metrics.** Numerical measurements aggregated over time: requests per second, latency percentiles, error counts, model load times, cache hit rates. Cheap, dense, suitable for dashboards and alerts.
2. **Logs.** Per-event records of what happened: "received query X for user Y, returned 10 results, took 23 ms." Verbose, useful for debugging individual incidents, expensive to store at scale.
3. **Traces.** A view of a single request's journey through multiple services, with timing for each hop. Useful for diagnosing where time is being spent or where errors originate. Often tied to distributed-tracing systems like Jaeger or OpenTelemetry.

A mature search system uses all three. DruSearch currently leans on metrics (Prometheus) with some logs; tracing is mentioned in the docs but is not heavily exercised on a single-machine demo.

Prometheus and Grafana

Prometheus is the de-facto standard open-source metrics system. It works on a pull model: services expose a `/metrics` endpoint with their counters and gauges, and a Prometheus server scrapes them every few seconds. Aggregations, alerts, and queries run against the resulting time-series database.

DruSearch's Go API exposes `/metrics` with a small set of measurements:

- Request counts and latencies per endpoint.
- Embedder call counts, latencies, and circuit-breaker state.
- OpenSearch query latencies.
- Model load events.
- Event-bus queue depth.

Grafana is the dashboarding tool typically paired with Prometheus. You build dashboards that show your metrics over time. DruSearch does not ship a Grafana setup, but a production deployment would have at least:

- A request-latency dashboard with p50, p95, p99 per endpoint.
- A search-quality dashboard with downstream click-through and conversion rates.
- A model-health dashboard with current model version, last reload time, and rerank latency.

What to alert on

Alerts are metrics that page someone when they cross a threshold. The art of alerting is keeping pages rare and meaningful.

A reasonable starting set for a search system:

- **Error rate above 1%.** Something is broken.
- **p99 latency above 200 ms** (or whatever your SLO is). User experience is degrading.
- **Model not loaded.** The serving binary has lost its model and is silently returning unranked results.
- **Embedder sustained failure.** Hybrid retrieval is degraded.
- **Event ingestion backed up.** Future training data is being lost.
- **Sustained drop in click-through rate.** Search quality has regressed in production.

Notice that several of these are *quality* alerts, not infrastructure alerts. The infra is fine — the requests are completing — but the results are bad. Catching quality regressions is much harder than catching crashes, and most teams underinvest here.

Logs as a debugging tool

Metrics tell you something is wrong. Logs tell you what specifically. A useful pattern:

- Each request gets a **request ID** or **query ID**, propagated through every service it touches.
- Each log line includes that ID.
- When debugging a specific user complaint, you grep the logs by ID and reconstruct the entire trace.

DruSearch attaches a `query_id` to every search response and log line. When a user reports "I searched for X and got Y," you can `grep query_id=...` and see exactly what the system did.

Logs at scale are expensive. Sampling, retention policies, and selective verbosity (info vs. debug vs. trace) are part of the operational cost. For small systems, log everything; for large ones, log strategically.

Service-level objectives

A **service-level objective (SLO)** is a target for some user-visible metric, like "99% of search requests succeed within 100 ms." Setting and tracking SLOs forces a team to be honest about what good performance means. The related concepts:

- **SLI:** Service-Level Indicator. The actual metric being tracked.
- **SLA:** Service-Level Agreement. The contractual version, usually with consequences attached.
- **Error budget:** 1 - SLO. If your SLO is 99.9% availability, your error budget is 0.1%, or about 43 minutes per month. You can "spend" the error budget on risky deployments and feature launches.

Search systems with serious operational discipline define SLOs for:

- Availability (% of requests that succeed at all).
- Latency (% of requests within a target).
- Quality (often a downstream metric like click-through rate above a threshold).

DruSearch does not have formal SLOs because it is a demo. A production deployment of a system like this would. Defining them early forces discussions that are otherwise easy to avoid.

What good looks like

When observability is set up well:

- A new engineer can answer "is the system working right now?" in 30 seconds.
- A regression is detected by an alert before a user complains.

- An incident can be traced from "user reported X" to "the cause was Y" in minutes.
- A trend (slow degradation in click-through rate, growing tail latency) is visible weeks before it becomes a crisis.

When observability is set up poorly:

- The first sign of an outage is users complaining on Twitter.
- Debugging requires SSH-ing into production and running `top`.
- "Was it slower last week?" cannot be answered.
- A bad deploy goes unnoticed until something else breaks.

Observability is the unsexy work that makes everything else possible.

Part VI — Putting It Together

Chapter 20: The Full System

We have crossed every conceptual boundary the book set out to cover. The remaining work is to put the pieces back together as a single picture — a search system, end to end, as a sequence of interacting parts.

The cold path: building the index and the model

Before any user types a query, work has been done. The cold path produces the artifacts that the hot path serves from.

1. Catalog ingestion
Source: Amazon Shopping Queries / ESCI dataset
Output: Postgres `products` table, ESCI judgments
2. Lexical indexing
Read products, write the BM25 index in OpenSearch
3. Embedding
For each product, call the embedder sidecar to compute a 384-dim vector
Store the vector alongside the BM25 fields in OpenSearch
4. Behavioral simulation (or, in production, real events)
Generate impressions, clicks, purchases
Store in `search\_events` (Postgres)
5. Feature aggregation
Compute popularity priors, CTR priors per product → Postgres `product\_features`
Compute brand-affinity per user → Redis `feat:user:{id}`
6. Training row construction
For each (query, candidate product) pair, compute a feature vector and a label
Labels come from ESCI judgments, optionally supplemented by BGE teacher scores
Store rows in `training\_rows`
7. Model training
LightGBM (or XGBoost) trains on `training\_rows` using LambdaRank
Logged to MLflow as a versioned experiment
8. Promotion gate
Compute NDCG@10 of the new model against ESCI judgments
Compare to RRF baseline; require non-regression
If gate passes: write the model to `models/ltr\_reranker.txt`
Hot-reload the API

Each step is independently testable, independently re-runnable, and decoupled from the others by the data it produces.

The hot path: handling a query

Now a user types "running shoes."

1. Request arrives at the Go API
GET /search?q=running+shoes&k=10
2. Embed the query
API → embedder sidecar → 384-dim vector
(If the sidecar fails, skip this and degrade to BM25-only)
3. Retrieve candidates in parallel
 - BM25 search over OpenSearch: top 100 by lexical match
 - kNN search over OpenSearch: top 100 by vector similarity(If the user query had implied gender, soft-boost matching genders)
4. Fuse with RRF
Combine the two lists into a single ranked list of ~100-200 candidates
5. Build feature vectors
For each candidate:
 - Retrieval features (BM25 score/rank, kNN score/rank, RRF score)
 - Static product features (popularity prior, price log, etc.)
 - Interaction features (brand match, color match, query coverage, etc.)
 - User features (brand affinity from Redis, if user_id provided)29 features in total, in a fixed schema
6. Score with the LTR model
For each candidate, run the LightGBM model on its feature vector
Sort by predicted score
Take the top k
7. Build the response
Each result has a product_id, score, and explain object
Includes query_id, session_id, mode (hybrid+ltr), model_version
8. Log impressions
Async: queue impression events for the displayed results
Background worker writes them to Postgres
9. Return to user
~25 ms total latency

Every step has a fallback. Every external call has a timeout. Every model is versioned. Every event is logged.

The feedback loop

After the user receives the page:

1. User sees impressions

Already logged

2. User clicks a result → POST /events
API queues the click event
Background worker writes it to Postgres
3. User purchases (or doesn't) → POST /events again
Same path
4. Periodically, batch jobs run:
 - Recompute popularity priors (new event data → new aggregates)
 - Recompute CTR priors
 - Recompute per-user brand affinity
 - Optionally retrain the ranker
5. New aggregates are written to Postgres / Redis
6. New ranker is gated and promoted
7. Next user benefits from the updated system

This is the loop that makes the search system get better over time. It is also the loop that, if you are not careful, can degenerate into the position-bias collapse from Chapter 10. The ESCI labels and BGE teacher scores live outside the loop precisely to keep it grounded in objective relevance.

What this book has and has not covered

We have covered:

- Lexical and semantic retrieval, and their hybrid combination.
- Embeddings and vector search.
- Feature engineering and the parity problem.
- Labels, including the dangers of using clicks naively.
- Learning to rank with gradient-boosted trees.
- Teacher-student distillation.
- Personalization with light user features and graceful cold-start.
- Offline evaluation with NDCG and lift over baselines.
- Online operation: latency budgets, circuit breakers, graceful degradation.
- Model lifecycle: registry, gates, promotion, rollback.
- Observability: metrics, logs, alerts, SLOs.

We have not covered, or covered only briefly:

- **Query understanding at depth.** Spell correction, query rewriting, intent classification, multi-language support.
- **Diversification.** Ensuring the top results are not all near-duplicates.
- **Faceted search and filtering UIs.** The search box is one input; checkboxes and sliders are others.
- **Conversational and agentic search.** Question-answering systems, retrieval-augmented generation (RAG) with large language models, multi-turn dialog.
- **Multi-modal search.** Searching across images, video, audio, code.
- **Learned retrievers.** Models like ColBERT or learned sparse retrievers (SPLADE) that bridge lexical and dense retrieval more elegantly.
- **A/B testing methodology.** The full statistical machinery of experimentation.
- **Privacy, fairness, and trust.** Touched in Chapter 14 but each is a book of its own.

These omissions are deliberate. A first principles tour of search needs limits, and the parts we did cover are

the parts that almost every production search system has in some form. Once those are clear, the omitted topics become extensions and refinements rather than fundamental new ideas.

A final thought

The thing that makes search hard is not any one of the techniques in this book. It is the interaction. A change to the embedding model affects retrieval which affects the candidate set which affects the features which affects the ranker which affects what users click which affects the next training set. Every component touches every other component, often through subtle paths.

The discipline of search engineering is not mastering a single technique. It is keeping the system honest as all those interactions evolve: feature parity, evaluation gates, observability, careful labels, graceful degradation, versioned artifacts. Each of those is a small piece of insurance against the system fooling itself or its operators.

If you take one idea away from this book, take this: **a search system gets better when its feedback loop is grounded in something outside itself**. Click data alone teaches the system to copy itself. Human judgments, teacher models, randomized exploration, holdout evaluations — these are the things that ground the loop in reality. Without them, no amount of clever ML will converge to a better answer. With them, even simple techniques compound over time.

Build the loop. Ground it. Watch it run.

— *End*

Glossary

- **A/B test**. An experiment where users are randomly assigned to variants and outcomes are compared.
- **ANN (Approximate Nearest Neighbors)**. Algorithms that find approximately closest vectors quickly, trading exactness for speed.
- **BM25 (Best Matching 25)**. The standard lexical scoring function, an evolution of TF-IDF that handles term saturation and length normalization.
- **Cold start**. The problem of producing useful output for new users or items with no history.
- **Cosine similarity**. A measure of similarity between two vectors, equal to their dot product divided by the product of their magnitudes; ranges from -1 to 1.
- **CTR (Click-Through Rate)**. Fraction of impressions that resulted in a click.
- **Distillation**. Training a small (student) model to imitate a large (teacher) model's outputs.
- **Embedding**. A vector representation of an object (text, image, etc.) where similar objects have nearby vectors.
- **ESCI (Exact, Substitute, Complement, Irrelevant)**. Amazon's relevance grading scheme for product search.
- **Feature**. A single numeric input to a machine-learning model.
- **GBDT (Gradient-Boosted Decision Trees)**. A family of ML algorithms that builds many small decision trees in sequence.
- **HNSW (Hierarchical Navigable Small World)**. A widely-used ANN algorithm that builds a hierarchical graph for fast nearest-neighbor search.
- **Inverted index**. A data structure mapping each term to the list of documents containing it.
- **kNN (k-Nearest Neighbors)**. The operation of finding the k closest items to a query.
- **LTR (Learning to Rank)**. The family of ML techniques for ranking items in response to queries.
- **LambdaMART / LambdaRank**. Pairwise (with list-aware reweighting) LTR algorithms; Light-GBM's default for ranking.
- **MLflow**. An open-source platform for ML lifecycle management.
- **NDCG (Normalized Discounted Cumulative Gain)**. The most common ranking quality metric, with a discount on lower positions and graded relevance support.

- **Online vs. offline.** Online = the live request path; offline = batch and training pipelines.
- **Position bias.** The tendency for top-ranked results to receive disproportionate engagement regardless of true relevance.
- **Prior.** A baseline estimate of a quantity, often used as a feature when stronger evidence is missing.
- **QPS (Queries Per Second).** A measure of system throughput.
- **Recall.** Fraction of relevant items found.
- **Retrieval vs. ranking.** Retrieval finds candidates fast (optimized for recall); ranking orders them carefully (optimized for precision).
- **RRF (Reciprocal Rank Fusion).** A simple, scale-free way to combine multiple ranked lists.
- **Sample weight.** A multiplier on how much a training example influences the model.
- **SLO (Service-Level Objective).** A target for a service-level metric like availability or latency.
- **Smoothing.** Adjusting an estimate (often a ratio) by mixing in a prior to reduce noise from small samples.
- **TF-IDF (Term Frequency, Inverse Document Frequency).** The classical lexical scoring function that BM25 evolved from.
- **Vector.** A list of numbers, used here to represent items as points in a high-dimensional space.

